Manual Técnico de sotOS

Un Micronúcleo Verificable con
Bypass Selectivo de Exonúcleo

Versión 1.0 — Marzo 2026

Santiago Sotomayor

Objetivo de diseño: $\leq 10\,000$ líneas de código en el núcleo
Cinco primitivas: planificación, virtualización de IRQ, IPC, capacidades, asignación de marcos

Índice general

I Fundamentos **1**

1. Introducción y Filosofía de Diseño **3**
1.1. Motivación . . . 3
1.2. Principios de Diseño . . . 3
1.2.1. Base Computacional de Confianza Mínima (TCB) . . . 3
1.2.2. Seguridad Basada en Capacidades . . . 4
1.2.3. Bypass Selectivo de Exonúcleo . . . 4
1.3. Comparación Arquitectónica . . . 4
1.4. Arquitectura General del Sistema . . . 5
1.5. Estructura del Código Fuente . . . 5
1.6. Notación y Convenciones . . . 6

2. Sistema de Capacidades **7**
2.1. Definiciones Formales . . . 7
2.2. Pool Genérico con Contadores de Generación . . . 8
2.2.1. El Problema ABA . . . 8
2.2.2. Codificación de Handles . . . 8
2.2.3. Operaciones del Pool . . . 9
2.2.4. Corrección de la Protección ABA . . . 10
2.3. Tabla de Capacidades . . . 10
2.3.1. Validación . . . 10
2.4. Delegación de Capacidades . . . 11
2.5. Árbol de Derivación de Capacidades (CDT) . . . 11
2.5.1. Algoritmo de Revocación . . . 12
2.6. Tipos de Objetos y su Semántica . . . 13
2.7. Invariantes del Sistema de Capacidades . . . 14

3. Comunicación Entre Procesos (IPC) **15**
3.1. Endpoints Síncronos . . . 15
3.1.1. Modelo Formal . . . 15
3.1.2. Pools por CPU . . . 16
3.1.3. Semántica de Rendezvous . . . 16
3.1.4. Propiedades del Protocolo . . . 19
3.1.5. Mecanismo de Timeout . . . 20
3.2. Canales Asíncronos . . . 20
3.3. Transferencia de Capacidades por IPC . . . 21
3.4. Resumen de Complejidades . . . 21

4. Planificación **23**
4.1. Multi-Level Queue (MLQ) . . . 23
4.1.1. Estructura de Colas . . . 23
4.1.2. Algoritmo de Despacho . . . 23
4.2. Robo de Trabajo (Work Stealing) . . . 24
4.2.1. Algoritmo . . . 24
4.2.2. Análisis de Rendimiento . . . 25
4.3. Cambio de Contexto . . . 25
4.3.1. Salvar y Restaurar Registros . . . 25
4.3.2. Transición a Modo Usuario: SYSCALL/SYSRET . . . 26
4.3.3. Restauración de TLS y Estado Extendido . . . 27
4.4. Dominios de Planificación . . . 27
4.5. Límites de Recursos por Hilo . . . 27
4.6. Asignación de Marcos Físicos . . . 28
4.6.1. Asignador de Bitmap . . . 28
4.6.2. Inicialización desde el Mapa de Memoria . . . 30
4.7. Resumen del Capítulo . . . 30

II Espacio de Usuario **31**

5. Gestión de Memoria **33**
5.1. Asignador de Marcos Físicos . . . 33
5.1.1. Estructura de Datos . . . 33
5.1.2. Algoritmo de Asignación . . . 34
5.1.3. Asignación Contigua para DMA . . . 34
5.1.4. Tabla de Refcount para CoW . . . 35
5.2. Tablas de Páginas: x86-64 de Cuatro Niveles . . . 35
5.2.1. Estructura de Traducción . . . 35
5.2.2. Extracción de Índices . . . 36
5.2.3. Algoritmos de Tabla de Páginas . . . 37
5.3. Copy-on-Write (CoW) . . . 38
5.3.1. Algoritmo de Clonación CoW . . . 39
5.3.2. Manejo de Fallos CoW en el VMM . . . 40
5.4. Enforcement W^X via mprotect . . . 41

6. LUCAS — Capa de Compatibilidad Linux **43**
6.1. Arquitectura del Redireccionamiento de Syscalls . . . 43
6.1.1. Recorrido Completo de una Syscall: `write(1, "Hola", 4)` . . . 43
6.2. Emulación de la ABI Linux: 127+ Syscalls . . . 44
6.3. Modelo de Procesos: Fork/Exec/Wait . . . 44
6.3.1. Grupos de Hilos (`CLONE_THREAD`) . . . 44
6.3.2. Fork con CoW . . . 44
6.3.3. Entrega de Señales . . . 45
6.4. Puente al Sistema de Archivos Virtual . . . 46
6.4.1. ObjectStore: Almacenamiento Transaccional . . . 46
6.4.2. Resolución de Rutas con CWD . . . 46
6.4.3. Snapshots CoW . . . 46

6.5. vDSO: Shared Library Forjado 46

7. Servicios del Sistema **49**
7.1. Servicio de Red (smoltcp) 49
7.1.1. Interfaz de Comandos IPC 49
7.1.2. Flujo de una Transferencia TCP 50
7.1.3. Transferencia TCP en Bloques 50
7.1.4. Resolución DNS por UDP 50
7.2. Servicio VMM (Virtual Memory Manager) 50
7.2.1. Manejo de Fallos de Página 50
7.2.2. Registro Global de Fallos 51
7.3. Servicio de Teclado 51
7.4. ObjectStore y VFS 51
7.4.1. Layout en Disco 51
7.4.2. Entrada de Directorio 51
7.4.3. Write-Ahead Logging 52

8. Arranque y Configuración **55**
8.1. Bootloader: Limine (BIOS, GPT+FAT32) 55
8.2. Secuencia de Arranque del Kernel 55
8.3. Bootstrapping del Proceso Init 56
8.4. Layout del Espacio de Direcciones 57
8.5. Configuración de QEMU 57
8.6. Resumen de la Parte II 58

Parte I

Fundamentos

Capítulo 1

Introducción y Filosofía de Diseño

1.1. Motivación

La historia de los sistemas operativos está marcada por una tensión fundamental entre *rendimiento* y *seguridad*. Los núcleos monolíticos como Linux incorporan millones de líneas de código en el espacio privilegiado (Ring 0), incluyendo controladores de dispositivos, sistemas de archivos y pilas de red. Cada línea de código en Ring 0 es una línea que, ante un error, puede comprometer la integridad completa del sistema.

SOTOS adopta una posición radical: *el código en Ring 0 debe ser mínimo y verificable*. Con un objetivo de $\leq 10\,000$ líneas de código (LOC) en el núcleo, el sistema implementa exactamente cinco primitivas fundamentales:

1. **Planificación**: Colas multinivel (MLQ) con robo de trabajo entre CPUs.
2. **Virtualización de IRQ**: Delegación de interrupciones a espacio de usuario.
3. **IPC**: Endpoints síncronos (estilo L4) y canales asíncronos (ring buffers).
4. **Capacidades**: Seguridad basada en tokens opacos con derechos asociados.
5. **Asignación de marcos**: Bitmap de marcos físicos de 4 KiB.

Todo lo demás — memoria virtual (paginación), sistema de archivos, controladores, pila de red — se ejecuta en espacio de usuario como servidores independientes.

1.2. Principios de Diseño

1.2.1. Base Computacional de Confianza Mínima (TCB)

Definición 1.1 (Base Computacional de Confianza)**.** La *Base Computacional de Confianza* (TCB, por *Trusted Computing Base*) de un sistema es el conjunto mínimo de componentes de hardware, firmware y software cuyo funcionamiento correcto es suficiente para garantizar la política de seguridad del sistema.

En un núcleo monolítico, la TCB incluye todo el código del kernel, típicamente millones de LOC. En SOTOS, la TCB comprende únicamente:

- El micronúcleo ($\leq 10\,000$ LOC).

- El hardware (CPU x86_64, controlador de memoria).
- El cargador de arranque (Limine v8).

Teorema 1.1 (Reducción de superficie de ataque)**.** *Sea n_M el número de líneas de código del kernel monolítico y n_μ el del micronúcleo, con $n_\mu \ll n_M$. Si la tasa de defectos es λ defectos por línea (constante), entonces el número esperado de defectos en la TCB se reduce de λn_M a λn_μ, logrando una reducción proporcional a n_μ/n_M.*

Demostración. Sea $D(n) = \lambda n$ el número esperado de defectos en n líneas de código, donde $\lambda > 0$ es la densidad de defectos. Entonces:

$$\frac{D(n_\mu)}{D(n_M)} = \frac{\lambda n_\mu}{\lambda n_M} = \frac{n_\mu}{n_M}.$$

Para sotOS con $n_\mu \leq 10\,000$ y un kernel Linux típico con $n_M \approx 30\,000\,000$:

$$\frac{n_\mu}{n_M} \leq \frac{10\,000}{30\,000\,000} \approx 3{,}3 \times 10^{-4}.$$

Esto implica una reducción de más de tres órdenes de magnitud en el número esperado de defectos dentro de la TCB. □

1.2.2. Seguridad Basada en Capacidades

A diferencia de los modelos de control de acceso discrecional (DAC) o mandatorio (MAC) que asocian permisos a identidades o etiquetas, sotOS emplea un modelo de *capacidades* (*capabilities*): tokens opacos que simultáneamente identifican un objeto del kernel y codifican los derechos de acceso sobre él.

Definición 1.2 (Capacidad)**.** Una *capacidad* es un par $c = (o, r)$ donde $o \in \mathcal{O}$ es un objeto del kernel y $r \in 2^{\mathcal{R}}$ es un conjunto de derechos, con $\mathcal{R} = \{\textsc{Read}, \textsc{Write}, \textsc{Execute}, \textsc{Grant}, \textsc{Revoke}\}$.

Este modelo elimina las vulnerabilidades de tipo *confused deputy*, donde un servicio privilegiado es engañado para abusar de sus permisos, porque el derecho de acceso viaja con la referencia al objeto y no con la identidad del invocador.

1.2.3. Bypass Selectivo de Exonúcleo

Un exonúcleo expone directamente los recursos de hardware al espacio de usuario, eliminando abstracciones del kernel. sotOS adopta un enfoque híbrido: el micronúcleo provee abstracciones seguras (IPC, capacidades, planificación), pero ciertos caminos críticos de rendimiento permiten acceso directo a hardware mediado por capacidades de tipo `IoPort` y `Memory`.

Ejemplo 1.1. Un controlador de disco NVMe en espacio de usuario recibe una capacidad $c_{\text{mmio}} = (\text{Memory}(\texttt{0xFE000000}, 4096), \{\textsc{Read}, \textsc{Write}\})$ que le permite acceder directamente a los registros MMIO del dispositivo sin pasar por el kernel en cada operación de E/S.

1.3. Comparación Arquitectónica

sotOS se ubica entre el micronúcleo puro y el exonúcleo: provee las cinco primitivas del micronúcleo con la opción de bypass directo a hardware cuando la política de seguridad lo permite (vía capacidades `IoPort` y `Memory`).

Cuadro 1.1: Comparación de arquitecturas de sistemas operativos.

Propiedad	**Monolítico**	**Micronúcleo**	**Exonúcleo**
Código en Ring 0	Todo	Mínimo	Mínimo
TCB (LOC)	$\sim 10^7$	$\sim 10^4$	$\sim 10^3$
Abstracción de HW	Completa	Parcial	Ninguna
Rendimiento de IPC	N/A (in-kernel)	Crítico	N/A
Aislamiento de fallos	Ninguno	Por proceso	Por proceso
Controladores	En kernel	En usuario	En usuario
Verificabilidad	Impráctica	Factible	Factible

1.4. Arquitectura General del Sistema

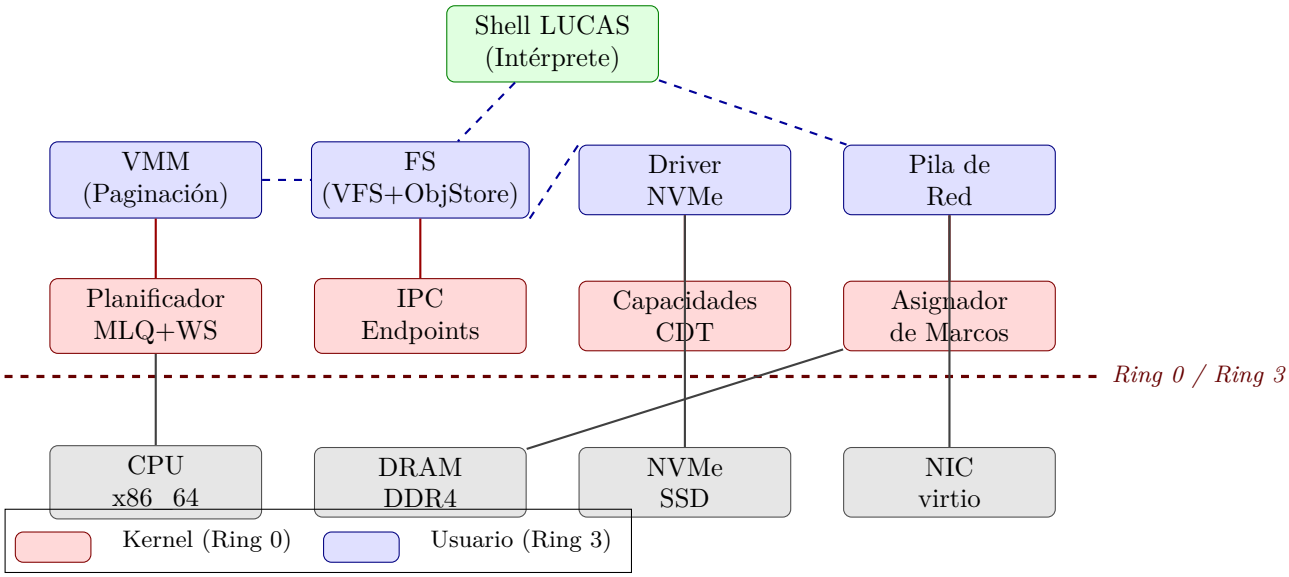

Figura 1.1: Arquitectura de capas de SOTOS: hardware, micronúcleo (Ring 0) y servidores de espacio de usuario (Ring 3). Las flechas continuas representan llamadas al sistema; las discontinuas representan IPC entre servidores.

1.5. Estructura del Código Fuente

El repositorio sigue una organización modular:

`kernel/` El micronúcleo (Ring 0):

- **`arch/x86_64/`** Código específico de CPU: serial, GDT, IDT, LAPIC.
- **`mm/`** Asignador de marcos físicos (bitmap).
- **`cap/`** Tabla de capacidades y CDT.
- **`ipc/`** Endpoints (síncronos) y canales (asíncronos).
- **`sched/`** Gestión de hilos y planificador.

`pool.rs` Asignador genérico `Pool<T>` con contadores de generación.

`libs/sotos-common/` ABI compartida kernel/usuario (syscalls, errores).

`services/` Servidores de usuario: VMM, FS, controladores, shell.

`boot/` Configuración del cargador Limine.

`scripts/` Herramientas de construcción (`mkimage.py`).

1.6. Notación y Convenciones

A lo largo de este manual empleamos la siguiente notación:

- $\mathcal{O}$ denota el conjunto de objetos del kernel.
- $\mathcal{R} = \{\mathrm{R}, \mathrm{W}, \mathrm{X}, \mathrm{G}, \mathrm{Rv}\}$ es el conjunto universal de derechos (Read, Write, Execute, Grant, Revoke).
- Para un entero x de n bits, $x[a{:}b]$ denota los bits desde la posición b (inclusive) hasta la posición a (inclusive), con el bit 0 siendo el menos significativo.
- $\lfloor x \rfloor_n$ denota x mód n.
- $|S|$ denota la cardinalidad de un conjunto finito S.
- Los fragmentos de código se presentan en Rust (el lenguaje de implementación).

Capítulo 2

Sistema de Capacidades

El sistema de capacidades es el pilar de seguridad de SOTOS. Cada objeto del kernel — marcos de memoria, endpoints de IPC, hilos, puertos de E/S — solo es accesible a través de una capacidad que simultáneamente identifica el objeto y codifica los derechos de acceso. No existe un mecanismo alternativo: ni identificadores de proceso, ni listas de control de acceso, ni anillos de protección adicionales. Las capacidades son *el único* mecanismo de autorización.

2.1. Definiciones Formales

Definición 2.1 (Universo de Objetos)**.** El *universo de objetos* $\mathcal{O}$ de SOTOS es la unión disjunta:

$$\mathcal{O} = \mathcal{O}_{\text{Mem}} \cup \mathcal{O}_{\text{Ep}} \cup \mathcal{O}_{\text{Ch}} \cup \mathcal{O}_{\text{Thr}} \cup \mathcal{O}_{\text{Irq}} \cup \mathcal{O}_{\text{Io}} \cup \mathcal{O}_{\text{Notif}} \cup \mathcal{O}_{\text{Dom}} \cup \mathcal{O}_{\text{AS}}$$

donde cada subconjunto corresponde a un tipo de objeto:

- $\mathcal{O}_{\text{Mem}} = \{(\text{base}, \text{size}) \mid \text{base}, \text{size} \in \mathbb{N}_{64}\}$: regiones de memoria física.
- $\mathcal{O}_{\text{Ep}} = \{e_i \mid i \in [0, C \cdot E)\}$: endpoints de IPC, donde $C =$ `MAX_CPUS` y $E =$ `MAX_EP_PER_CORE` $=$ 64.
- $\mathcal{O}_{\text{Ch}} = \{ch_i\}$: canales asíncronos.
- $\mathcal{O}_{\text{Thr}} = \{t_i\}$: hilos.
- $\mathcal{O}_{\text{Irq}} = \{\text{irq}_n \mid n \in [0, 256)\}$: líneas de interrupción.
- $\mathcal{O}_{\text{Io}} = \{(\text{base}, \text{count}) \mid \text{base} \in [0, 2^{16}), \text{count} \in [1, 2^{16}]\}$: rangos de puertos x86 de E/S.
- $\mathcal{O}_{\text{Notif}} = \{n_i\}$: objetos de notificación (semáforos binarios).
- $\mathcal{O}_{\text{Dom}} = \{d_i\}$: dominios de planificación.
- $\mathcal{O}_{\text{AS}} = \{(\text{cr3})\}$: espacios de direcciones (PML4).

Definición 2.2 (Derechos)**.** El conjunto de derechos es $\mathcal{R} = \{\text{R}, \text{W}, \text{X}, \text{G}, \text{Rv}\}$, codificado como un bitmask de 5 bits:

$$\text{R} = 2^0, \quad \text{W} = 2^1, \quad \text{X} = 2^2, \quad \text{G} = 2^3, \quad \text{Rv} = 2^4.$$

El derecho universal es $\mathcal{R}_{\text{ALL}} = \text{R} \mid \text{W} \mid \text{X} \mid \text{G} \mid \text{Rv} =$ `0x1F`. Un subconjunto de derechos $r \subseteq \mathcal{R}$ se representa como un entero $r \in [0, 31]$.

Definición 2.3 (Capacidad)**.** Una *capacidad* es una terna $c = (o, r, p)$ donde:

- $o \in \mathcal{O}$ es el objeto referenciado.
- $r \in 2^{\mathcal{R}}$ es el conjunto de derechos (bitmask de 5 bits).
- $p \in \{\perp\} \cup \mathcal{C}$ es la capacidad padre en el árbol de derivación ($\perp$ indica capacidad raíz).

donde $\mathcal{C}$ es el conjunto de todas las capacidades existentes en el sistema.

2.2. Pool Genérico con Contadores de Generación

Antes de describir la tabla de capacidades, definimos la estructura de datos subyacente que proporciona asignación $\mathcal{O}(1)$ con protección contra el problema ABA.

2.2.1. El Problema ABA

Definición 2.4 (Problema ABA)**.** En un sistema con reciclaje de identificadores, el *problema ABA* ocurre cuando:

1. Un actor obtiene una referencia al objeto A en el slot i.
2. El objeto A es liberado del slot i.
3. Un nuevo objeto B es asignado al slot i.
4. El actor usa su referencia obsoleta, que ahora apunta a B en lugar de A.

En el contexto de un kernel, este problema es particularmente peligroso: si un proceso mantiene un `CapId` cuyo slot fue reciclado, podría inadvertidamente acceder a un objeto completamente diferente con derechos que no le corresponden.

2.2.2. Codificación de Handles

SOTOS resuelve el problema ABA mediante *contadores de generación* empaquetados en cada handle.

Definición 2.5 (PoolHandle)**.** Un `PoolHandle` es un entero sin signo de 32 bits con la siguiente codificación:

$$h = (g \ll 20) \mid s$$

donde:

- $s = h[19{:}0]$ es el *índice de slot*, con $s \in [0, 2^{20})$ (máximo 1 048 575 slots).
- $g = h[31{:}20]$ es el *contador de generación*, con $g \in [0, 2^{12})$ (wrap-around a 4096).

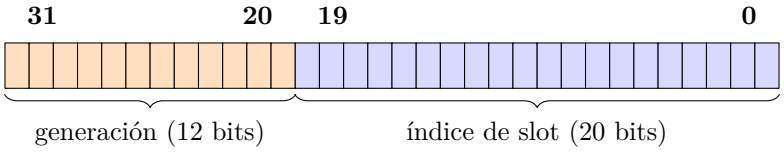

Figura 2.1: Codificación de un `PoolHandle`: los 12 bits superiores almacenan el contador de generación y los 20 bits inferiores el índice del slot.

2.2.3. Operaciones del Pool

El `Pool<T>` soporta tres operaciones fundamentales:

Algoritmo 1: POOL-ALLOC(P, v): Asignar un valor v en el pool P.

Input: Pool P, valor v de tipo T
Output: Handle h con generación correcta

```
1 if P.free_list ≠ ∅ then
2     i ← P.free_list.POP();
3     P.slots[i] ← SOME(v);
4     return PACK(i, P.gens[i]);
5 i ← |P.slots|;
6 P.slots.PUSH(SOME(v));
7 P.gens.PUSH(0);
8 return PACK(i, 0);
```

Algoritmo 2: POOL-FREE(P, h): Liberar el slot referenciado por h.

Input: Pool P, handle h
Output: SOME(v) si el handle era válido, NONE en caso contrario

```
1 (i, g) ← UNPACK(h);
2 if i ≥ |P.slots| or P.gens[i] ≠ g then
3     return NONE;                          // Handle obsoleto o fuera de rango
4 if P.slots[i] = NONE then
5     return NONE;                                                // Doble liberación
6 v ← P.slots[i].TAKE();
7 P.gens[i] ← (P.gens[i] + 1) mód 2^12;
8 P.free_list.PUSH(i);
9 return SOME(v);
```

Algoritmo 3: POOL-GET(P, h): Buscar el valor referenciado por h.

Input: Pool P, handle h
Output: SOME$(\&v)$ si válido, NONE en caso contrario

```
1 (i, g) ← UNPACK(h);
2 if i ≥ |P.slots| or P.gens[i] ≠ g then
3     return NONE;
4 return P.slots[i].ASREF();
```

2.2.4. Corrección de la Protección ABA

Teorema 2.1 (El contador de generación previene uso-después-de-liberar)**.** *Sea P un `Pool<T>` y sea $h = \textsc{Pack}(i, g)$ un handle obtenido de* Pool-Alloc*. Si el slot i es liberado y reasignado k veces antes de que h sea usado en* Pool-Get*, entonces h será rechazado siempre que $k \not\equiv 0 \pmod{2^{12}}$.*

Demostración. Cada invocación de Pool-Free incrementa $P.\text{gens}[i]$ módulo 2^{12}. Después de k liberaciones, la generación almacenada es:

$$g' = (g + k) \bmod 2^{12}.$$

La verificación en Pool-Get compara g (la generación del handle) con g' (la generación actual del slot). La comparación falla si y solo si $g \neq g'$, es decir, $k \not\equiv 0 \pmod{2^{12}}$.

Para $G_{\text{máx}} = 2^{12} = 4096$, un handle obsoleto será correctamente rechazado en las primeras 4095 reasignaciones del mismo slot. Solo si el slot es reciclado exactamente un múltiplo de 4096 veces entre la obtención y el uso del handle, este sería incorrectamente aceptado — una condición estadísticamente negligible en la práctica. □

Corolario 2.2 (Probabilidad de colisión ABA)**.** *Si los accesos a handles obsoletos son uniformemente aleatorios con respecto al número de reciclajes del slot, la probabilidad de que un handle obsoleto sea aceptado es $1/G_{\text{máx}} = 1/4096 \approx 2{,}44 \times 10^{-4}$ por intento.*

Observación 2.1. En la práctica, la ventana de vulnerabilidad es aún menor porque: (a) los slots se reciclan mediante una lista libre LIFO, distribuyendo la reutilización entre todos los slots disponibles; y (b) un proceso que intenta usar un handle obsoleto típicamente lo hace después de pocas reasignaciones, no miles.

2.3. Tabla de Capacidades

La tabla de capacidades es la estructura central de seguridad del kernel. Almacena todas las capacidades del sistema, protegida por un spinlock global.

Definición 2.6 (Tabla de Capacidades)**.** La *tabla de capacidades* $\mathcal{T}$ es un `Pool<CapEntry>` donde cada entrada es una terna (o, r, p) según la definition 2.3. Las operaciones son:

- Insert$(o, r, p) \to$ `CapId`: crear una nueva capacidad.
- Validate(`CapId`, r_{req}) $\to$ Result: verificar existencia y derechos suficientes.
- Lookup(`CapId`) $\to (o, r)$: obtener el objeto y derechos.
- Revoke(`CapId`): revocar la capacidad y todas sus derivadas.

2.3.1. Validación

La validación es el camino crítico de rendimiento: cada llamada al sistema que opera sobre un objeto del kernel primero valida la capacidad proporcionada por el espacio de usuario.

Algoritmo 4: CAP-VALIDATE($\mathcal{T}, h, r_{\text{req}}$)

Input: Tabla $\mathcal{T}$, handle h (del espacio de usuario), derechos requeridos r_{req}
Output: Objeto o si la validación tiene éxito, error en caso contrario

```
e ← POOL-GET(T, h);
if e = NONE then return ERR(InvalidCap);
if e.r & r_req ≠ r_req then
    return ERR(NoRights);                    // Derechos insuficientes
return OK(e.o);
```

Proposición 2.3 (Complejidad de la validación)**.** *CAP-VALIDATE ejecuta en $\mathcal{O}(1)$: una búsqueda indexada en el pool (verificación de generación + acceso a array) seguida de una operación bitwise AND.*

2.4. Delegación de Capacidades

La delegación (*grant*) permite a un proceso transferir acceso a un objeto a otro proceso, con derechos opcionalmente restringidos.

Definición 2.7 (Operación de Delegación)**.** Dada una capacidad fuente $c_s = (o, r_s, p_s)$ con $\mathrm{G} \in r_s$, y una máscara de derechos $m \in 2^{\mathcal{R}}$, la *delegación* produce una nueva capacidad:

$$c_d = (o, r_s \,\&\, m, c_s)$$

donde c_s se convierte en el padre de c_d en el Árbol de Derivación de Capacidades.

Lema 2.4 (Monotonía de Derechos)**.** *Para cualquier capacidad derivada c_d de una capacidad fuente c_s mediante delegación con máscara m:*

$$r_d = r_s \,\&\, m \subseteq r_s.$$

Es decir, una capacidad derivada nunca puede tener más derechos que su padre.

Demostración. Para cualquier bit $b \in \{0, 1, 2, 3, 4\}$:

$$r_d[b] = r_s[b] \,\&\, m[b] \leq r_s[b]$$

ya que $x \,\&\, y \leq x$ para todo $x, y \in \{0, 1\}$. Por lo tanto, $r_d \subseteq r_s$ como conjuntos de derechos. □

Corolario 2.5 (Cadena de Delegaciones)**.** *Si $c_0 \to c_1 \to \cdots \to c_k$ es una cadena de delegaciones, entonces $r_k \subseteq r_{k-1} \subseteq \cdots \subseteq r_0$. Los derechos son monótonamente no crecientes a lo largo de la cadena.*

Demostración. Aplicar el theorem 2.4 inductivamente: si $r_i \subseteq r_{i-1}$ para todo $i \leq k$ (base: $r_1 = r_0 \,\&\, m_1 \subseteq r_0$), entonces $r_{i+1} = r_i \,\&\, m_{i+1} \subseteq r_i \subseteq r_{i-1}$. □

2.5. Árbol de Derivación de Capacidades (CDT)

El CDT registra la relación padre-hijo entre capacidades, permitiendo la revocación transitiva: al revocar una capacidad, se revocan automáticamente todas las capacidades derivadas de ella.

Definición 2.8 (CDT)**.** El *Árbol de Derivación de Capacidades* (CDT) es un bosque dirigido $G = (\mathcal{C}, E)$ donde:

- $\mathcal{C}$ es el conjunto de capacidades vivas.
- $(c_p, c_d) \in E$ si c_d fue creada por delegación desde c_p.
- Las capacidades raíz (insertadas directamente por el kernel) tienen $p = \perp$.

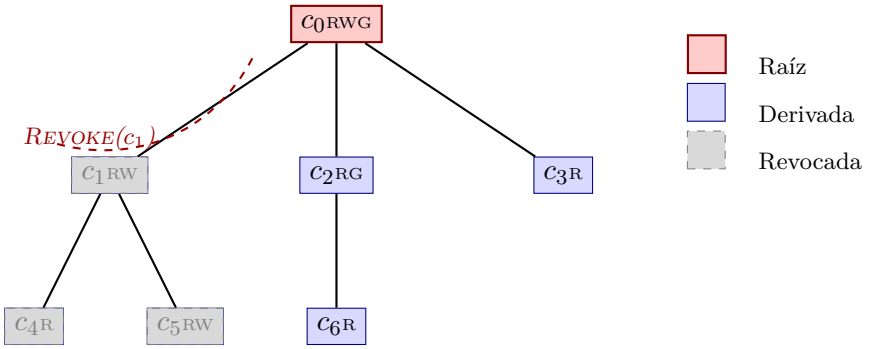

Figura 2.2: Árbol de Derivación de Capacidades. Al revocar c_1, se revocan transitivamente c_4 y c_5. Las capacidades c_2, c_3 y c_6 no se ven afectadas.

2.5.1. Algoritmo de Revocación

La revocación en sotOS se implementa mediante un algoritmo iterativo de punto fijo que elimina huérfanos hasta que no quedan más.

Algoritmo 5: Revoke($\mathcal{T}, h$): Revocar una capacidad y todas sus derivadas.

Input: Tabla $\mathcal{T}$, handle h de la capacidad a revocar

```
1 Pool-Free(T, h);                                  // Liberar la capacidad objetivo
2 repeat
3     orphans ← ∅;
4     foreach (h_i, e_i) ∈ T.iter() do
5         if e_i.p ≠ ⊥ and Pool-Get(T, e_i.p) = None then
6             orphans ← orphans ∪ {h_i};
7     foreach h_i ∈ orphans do
8         Pool-Free(T, h_i);
9 until orphans = ∅;
```

Teorema 2.6 (Terminación de la Revocación)**.** *El algoritmo* Revoke *termina en a lo sumo $d + 1$ iteraciones del bucle externo, donde d es la profundidad máxima del subárbol enraizado en la capacidad revocada.*

Demostración. Sea T_h el subárbol del CDT enraizado en h (incluyendo h). Definimos:

- $L_0 = \{h\}$ (nivel 0, la raíz).
- $L_k = \{c \in T_h \mid \text{dist}(h, c) = k\}$ (nivel k).

Iteración 0: Se libera h, lo que deja a todos los hijos directos L_1 como huérfanos (su padre h ya no existe en el pool).
Iteración k **(**$k \geq 1$**)**: El bucle identifica como huérfanos todos los nodos en L_k (sus padres en L_{k-1} fueron liberados en la iteración anterior). Se liberan todos los nodos de L_k.
Iteración $d+1$: Todos los nodos de L_d (el nivel más profundo) fueron liberados en la iteración d. No quedan huérfanos. El conjunto orphans es vacío y el bucle termina.

Como $d \leq |\mathcal{C}|$ (profundidad máxima acotada por el número total de capacidades, que a su vez está acotado por $S_{\text{máx}} = 2^{20}$), el algoritmo termina en tiempo finito. □

Teorema 2.7 (Corrección de la Revocación)**.** *Después de ejecutar* $\textsc{Revoke}(\mathcal{T}, h)$*, para todo* c *que era descendiente de* h *en el CDT (o* h *mismo),* $\textsc{Pool-Get}(\mathcal{T}, c) = \textsc{None}$*.*

Demostración. Por inducción sobre la profundidad k del nodo en el subárbol T_h.
Caso base ($k = 0$): h es liberado directamente en la primera línea del algoritmo.
Paso inductivo: Supongamos que todo nodo a profundidad $\leq k$ ha sido liberado. Sea c un nodo a profundidad $k+1$. Su padre está a profundidad k y, por hipótesis inductiva, ha sido liberado. Entonces $\textsc{Pool-Get}(\mathcal{T}, c.\text{parent}) = \textsc{None}$, por lo que c es identificado como huérfano en la siguiente iteración del bucle y es liberado.

Ningún nodo fuera de T_h es afectado porque: (a) los nodos raíz ($p = \perp$) nunca son identificados como huérfanos; (b) los nodos cuyo padre pertenece a otro subárbol mantienen su padre vivo. □

Proposición 2.8 (Complejidad de la Revocación)**.** *Sea* $n = |\mathcal{C}|$ *el número total de capacidades y* $m = |T_h|$ *el tamaño del subárbol revocado. La complejidad es* $\mathcal{O}(d \cdot n)$ *donde* d *es la profundidad del subárbol, o equivalentemente* $\mathcal{O}(n \cdot m)$ *en el peor caso (*$d = m$*, cadena lineal).*

Observación 2.2. Una implementación alternativa con punteros explícitos padre→hijos permitiría revocación en $\mathcal{O}(m)$, pero requeriría listas enlazadas dinámicas en cada entrada. sotOS prioriza la simplicidad del slot fijo para mantener el código del kernel dentro del objetivo de 10 000 LOC.

2.6. Tipos de Objetos y su Semántica

Cuadro 2.1: Tipos de objetos del kernel y operaciones válidas por derecho.

Tipo	**Variante Rust**	**Operaciones**
Memoria	`Memory{base, size}`	R: mapear como solo-lectura; W: mapear como lectura-escritura; X: mapear como ejecutable (W⊕X enforced)
Endpoint	`Endpoint{id}`	R: recv; W: send; G: transferir cap en IPC
Canal	`Channel{id}`	R: dequeue; W: enqueue
Hilo	`Thread{id}`	R: inspeccionar estado; W: modificar prioridad/afinidad; Rv: destruir
IRQ	`Irq{line}`	R: esperar interrupción; W: acknowledge
IoPort	`IoPort{base, count}`	R: `in`; W: `out`
Notificación	`Notification{id}`	R: wait; W: signal
Dominio	`Domain{id}`	R: consultar presupuesto; W: añadir/quitar miembros
Espacio Addr.	`AddrSpace{cr3}`	R: inspeccionar; W: map/unmap páginas

2.7. Invariantes del Sistema de Capacidades

Enunciamos los invariantes que el sistema de capacidades mantiene en todo momento.

Invariante 2.1 (Principio de Menor Privilegio). Todo acceso a un objeto del kernel requiere una capacidad válida con derechos suficientes. No existe un camino de acceso alternativo.

Invariante 2.2 (No-Amplificación). Para toda cadena de delegaciones $c_0 \to c_1 \to \cdots \to c_k$: $r_k \subseteq r_0$. Los derechos nunca aumentan por delegación.

Invariante 2.3 (Revocación Total). Al revocar c, todo descendiente de c en el CDT queda inaccesible. No se produce una "fuga" de capacidades.

Invariante 2.4 (Consistencia ABA). Si una capacidad c es liberada y su slot es reasignado a c', cualquier handle obsoleto que apuntaba a c será rechazado (con probabilidad $1 - 2^{-12}$) al acceder a c'.

Capítulo 3

Comunicación Entre Procesos (IPC)

La comunicación entre procesos es la operación más crítica de rendimiento en un micronúcleo. En un kernel monolítico, los subsistemas se comunican mediante llamadas a funciones internas; en un micronúcleo, cada interacción entre servidores requiere un cruce de dominios de protección. La eficiencia del IPC determina, por tanto, la viabilidad del diseño microkernel.

SOTOS provee cuatro mecanismos de IPC, cada uno optimizado para un patrón de uso:

1. **Endpoints síncronos**: Mensajes pequeños (72 bytes) transferidos en registros durante el cambio de contexto. Cero copias de memoria.
2. **Canales asíncronos**: Ring buffers acotados de 16 mensajes para transferencia de datos de alto throughput.
3. **Notificaciones**: Semáforos binarios para señalización ligera.
4. **Mailbox por CPU**: Colas de mensajes entre cores para entrega de IPC cross-core con IPI (Inter-Processor Interrupt).

3.1. Endpoints Síncronos

3.1.1. Modelo Formal

Definición 3.1 (Endpoint)**.** Un *endpoint* ϵ es una estructura de sincronización que se encuentra en uno de tres estados:

$$\epsilon.\text{state} \in \{\textsc{Idle}, \textsc{RecvWait}, \textsc{SendWait}\}$$

con los siguientes campos:

- $\epsilon.\text{receiver} \in \{\bot\} \cup \mathbb{T}$: hilo esperando recibir.
- $\epsilon.\text{send_queue} \in \mathbb{T}^*$: cola FIFO de hilos esperando enviar, con $|\epsilon.\text{send_queue}| \leq Q_{\text{máx}} = 16$.
- $\epsilon.\text{caller} \in \{\bot\} \cup \mathbb{T}$: hilo que ha enviado y espera respuesta (protocolo CALL).

donde $\mathbb{T}$ es el conjunto de identificadores de hilo.

Definición 3.2 (Mensaje IPC)**.** Un *mensaje* $m = (\text{tag}, \text{regs}, \text{cap})$ consiste en:

- $\text{tag} \in \mathbb{N}_{64}$: etiqueta que identifica la operación.

- regs $\in \mathbb{N}_{64}^{8}$: payload de 8 registros (512 bits = 64 bytes).
- cap $\in \{\perp\} \cup \mathbb{N}_{32}$: capacidad opcional a transferir.

El tamaño total es $8 + 64 + 4 = 76$ bytes (redondeado a 72 bytes por alineación).

3.1.2. Pools por CPU

A diferencia de un pool global (que sería un cuello de botella de contención en SMP), SOTOS mantiene un pool de endpoints independiente por cada core de CPU.

Definición 3.3 (Codificación de Handles de Endpoint)**.** Un handle de endpoint es un entero de 32 bits:

$$h_{\text{ep}} = (\text{core} \ll 28) \mid (g \ll 20) \mid s$$

donde:

- core $= h_{\text{ep}}[31{:}28]$: identificador de CPU (4 bits, máx. 16 cores).
- $g = h_{\text{ep}}[27{:}20]$: generación (8 bits, wrap-around a 256).
- $s = h_{\text{ep}}[19{:}0]$: índice de slot (20 bits, máx. 64 por core).

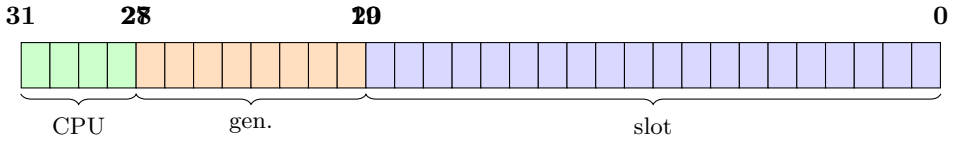

Figura 3.1: Codificación del handle de endpoint: 4 bits de core, 8 bits de generación, 20 bits de índice de slot.

3.1.3. Semántica de Rendezvous

El protocolo de endpoints síncronos implementa *rendezvous*: el emisor se bloquea hasta que un receptor esté listo (y viceversa). La transferencia del mensaje ocurre directamente entre los registros guardados de ambos hilos — sin buffer intermedio.

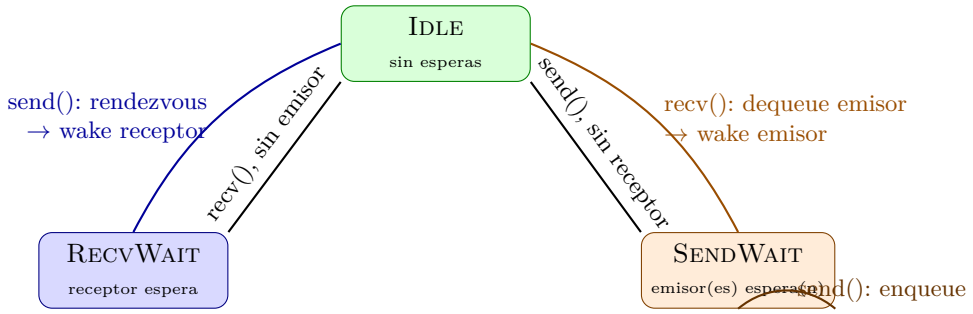

Figura 3.2: Máquina de estados de un endpoint síncrono. Las transiciones etiquetadas indican la operación que causa el cambio de estado y la acción resultante.

Operación Send

Algoritmo 6: ENDPOINT-SEND(ϵ, m, t_s): Enviar mensaje m desde hilo t_s.

Input: Endpoint ϵ, mensaje m, hilo emisor t_s
Output: OK si exitoso, error en caso contrario

```
// Verificar si hay un caller esperando respuesta
if ε.caller ≠ ⊥ then
    t_c ← ε.caller; ε.caller ← ⊥;
    Escribir m en registros de t_c;
    WAKE(t_c);
    return OK;                                  // Respuesta entregada
switch ε.state do
    case RECVWAIT do
        t_r ← ε.receiver; ε.receiver ← ⊥;
        ε.state ← IDLE;
        Escribir m en registros de t_r;
        WAKE(t_r);                              // Rendezvous: emisor no se bloquea
    case IDLE | SENDWAIT do
        if |ε.send_queue| = Q_máx then return ERR(Full);
        ε.send_queue.ENQUEUE(t_s);
        ε.state ← SENDWAIT;
        Guardar m en TCB de t_s;
        BLOCK(t_s);                             // Emisor se bloquea
return OK;
```

Operación Recv

Algoritmo 7: ENDPOINT-RECV(ϵ, t_r): Recibir mensaje para hilo t_r.

```
Input: Endpoint ϵ, hilo receptor t_r
Output: Mensaje m
switch ϵ.state do
    case SendWait do
        t_s ← ϵ.send_queue.Dequeue();
        is_caller ← (t_s & CALLER_BIT) ≠ 0;
        t_s ← t_s & ¬CALLER_BIT;
        m ← leer mensaje guardado en TCB de t_s;
        if is_caller then
            ϵ.caller ← t_s;                          // Caller espera respuesta
        else
            Wake(t_s);                               // Send puro: emisor libre
        if |ϵ.send_queue| = 0 then ϵ.state ← Idle;
        return m;
    case Idle do
        ϵ.receiver ← t_r;
        ϵ.state ← RecvWait;
        Block(t_r);                                  // Receptor se bloquea
        m ← leer mensaje de registros de t_r;
        return m;
    case RecvWait do
        return Err(AlreadyWaiting);                  // Solo un receptor
```

Operación Call (RPC)

Algoritmo 8: ENDPOINT-CALL(ϵ, m, t_c): Enviar m y esperar respuesta.

```
Input: Endpoint ϵ, mensaje m, hilo caller t_c
Output: Mensaje de respuesta m_r
switch ϵ.state do
    case RecvWait do
        t_r ← ϵ.receiver; ϵ.receiver ← ⊥;
        ϵ.caller ← t_c;
        ϵ.state ← Idle;
        Escribir m en registros de t_r;
        Wake(t_r);
        Block(t_c);                                  // Caller espera respuesta
    case Idle | SendWait do
        ϵ.send_queue.Enqueue(t_c | CALLER_BIT);
        ϵ.state ← SendWait;
        Guardar m en TCB de t_c;
        Block(t_c);                  // Caller espera tanto envío como respuesta
m_r ← leer respuesta de registros de t_c;
return m_r;
```

3.1.4. Propiedades del Protocolo

Teorema 3.1 (Transferencia Sin Copia)**.** *En el camino rápido (rendezvous inmediato), la operación* SEND *transfiere el mensaje directamente escribiendo en los registros guardados del receptor. No se realiza ninguna copia a memoria intermedia.*

Demostración. Cuando SEND encuentra el endpoint en estado RECVWAIT, la función WRITE_IPC_MSG(t_r, m) escribe los campos `tag` y `regs` del mensaje directamente en el campo `ipc_msg` del TCB (Thread Control Block) de t_r. Al despertar t_r y ejecutar su retorno de RECV, lee los datos de su propio TCB.

El TCB reside en memoria del kernel (heap del pool de hilos). La escritura es una asignación de struct — no una copia de páginas ni una llamada a `memcpy` de usuario. La ruta completa es: emisor→TCB-receptor, sin buffer auxiliar. □

Teorema 3.2 (Acotación de la Cola de Envío)**.** *La cola de envío de cada endpoint tiene cardinalidad máxima* $Q_{\text{máx}} = 16$*. Intentos adicionales de envío retornan* ERR(*OutOfResources*)*.*

Demostración. Por inspección del algoritmo ENDPOINT-SEND: antes de encolar, se verifica $|\epsilon.\text{send_queue}| < Q_{\text{máx}}$. El array subyacente es de tamaño fijo `[Option<ThreadId>; 16]`. □

Teorema 3.3 (Wait-Free en el Camino Rápido)**.** *Si el endpoint está en estado* RECVWAIT *al momento de* SEND *(o en* SENDWAIT *al momento de* RECV*), la operación completa en* $\mathcal{O}(1)$ *sin bloquear al invocador.*

Demostración. En ambos casos, la operación realiza:

1. Adquirir el lock del pool por-core: $\mathcal{O}(1)$ amortizado (ticket mutex FIFO, contención limitada a operaciones en el mismo core).

2. Leer/escribir campos del endpoint: $\mathcal{O}(1)$.

3. Soltar el lock.
4. Escribir mensaje en TCB del peer: $\mathcal{O}(1)$.
5. Despertar al peer: $\mathcal{O}(1)$ (enqueue en cola de ejecución).

El invocador no se bloquea en ningún punto. La operación es wait-free (no solo lock-free) para el hilo invocador, ya que completa en un número acotado de pasos sin depender del progreso de otros hilos. □

3.1.5. Mecanismo de Timeout

Definición 3.4 (IPC con Timeout)**.** Una operación Call-Timeout(ϵ, m, N) establece un plazo absoluto $\delta = \text{global_ticks} + N$. Si el hilo sigue bloqueado cuando global_ticks $\geq \delta$, el kernel:

1. Marca el flag `ipc_timed_out` en el TCB.
2. Despierta al hilo.
3. El hilo detecta el timeout y ejecuta Cancel-Caller para limpiar su estado en el endpoint.

Teorema 3.4 (Garantía de Espera Acotada)**.** *Para toda invocación de* Call-Timeout(ϵ, m, N) *con $N > 0$, el hilo invocador se desbloquea en a lo sumo N ticks del temporizador global, independientemente del comportamiento del servidor.*

Demostración. Sea t_0 el tick global al momento de la invocación. Se establece $\delta = t_0 + N$. El temporizador LAPIC genera interrupciones periódicas que incrementan `global_ticks` atómicamente. En la rutina del tick del planificador:

1. Para cada hilo bloqueado con `ipc_timeout` > 0:
2. Si `global_ticks` $\geq$ `ipc_timeout`:
3. Marcar `ipc_timed_out` $\leftarrow$ `true`.
4. Llamar Wake(t).

Como `global_ticks` es monótonamente creciente (AtomicU64 con fetch_add) y el timer tick ocurre con frecuencia f (100 Hz por defecto), el tiempo de pared hasta el timeout es $\leq N/f + 1/f$ segundos (un tick de latencia adicional por granularidad). El hilo se desbloquea garantizadamente. □

3.2. Canales Asíncronos

Definición 3.5 (Canal)**.** Un *canal* es un ring buffer acotado de capacidad $K = 16$ mensajes, con semántica producteur-consommateur:

- Ch-Send: encola un mensaje. Bloquea si está lleno.
- Ch-Recv: desencola un mensaje. Bloquea si está vacío.

Proposición 3.5 (Propiedades del Canal)**.** *Sea C un canal con capacidad K. Entonces:*

1. $0 \leq |C| \leq K$ *en todo momento (acotación).*

2. *Los mensajes se entregan en orden FIFO (ring buffer circular).*

3. *En ausencia de bloqueo,* CH-SEND *y* CH-RECV *son* $\mathcal{O}(1)$.

Demostración.

1. La cola empieza con $|C| = 0$. CH-SEND incrementa $|C|$ solo si $|C| < K$ (bloquea en caso contrario). CH-RECV decrementa $|C|$ solo si $|C| > 0$ (bloquea en caso contrario). Ambas operaciones están protegidas por un mutex, por lo que no hay carreras.

2. El ring buffer usa índices `head` y `tail` con aritmética modular: $\texttt{tail} = (\texttt{tail} + 1) \bmod K$ y $\texttt{head} = (\texttt{head} + 1) \bmod K$. El consumidor siempre lee desde `head` (el mensaje más antiguo).

3. Acceso indexado al array $\texttt{buffer}[i]$ es $\mathcal{O}(1)$. La aritmética modular es $\mathcal{O}(1)$.

□

3.3. Transferencia de Capacidades por IPC

Una característica fundamental del sistema es la capacidad de transferir capacidades entre procesos a través de mensajes IPC.

Definición 3.6 (Transferencia de Capacidad)**.** Cuando un mensaje m incluye $m.\text{cap} = c_s$ (handle de una capacidad fuente), el kernel automáticamente:

1. Verifica que c_s tenga el derecho GRANT.

2. Crea una nueva capacidad $c_d = \text{GRANT}(c_s, \mathcal{R}_{\text{ALL}})$ como hija de c_s.

3. Reemplaza $m.\text{cap}$ con el handle de c_d.

El receptor obtiene una capacidad derivada completa (todos los derechos del padre).

Proposición 3.6 (Seguridad de la Transferencia)**.** *La transferencia de capacidades por IPC preserva todos los invariantes del sistema:*

1. No-amplificación*: c_d tiene derechos $\leq c_s$ (por theorem 2.4).*

2. Revocabilidad*: c_d es hija de c_s en el CDT, por lo que revocar c_s revocará c_d (por theorem 2.7).*

3. Autenticidad*: Solo el poseedor de c_s (con derecho* GRANT*) puede iniciar la transferencia.*

3.4. Resumen de Complejidades

Cuadro 3.1: Complejidad temporal de las operaciones de IPC.

Operación	**Camino rápido**	**Camino lento**	**Notas**
SEND (rendezvous)	$\mathcal{O}(1)$	$\mathcal{O}(1)$ + bloqueo	Wait-free si receptor listo
RECV (rendezvous)	$\mathcal{O}(1)$	$\mathcal{O}(1)$ + bloqueo	Wait-free si emisor listo
CALL (RPC)	$\mathcal{O}(1)$	$\mathcal{O}(1)$ + 2 bloqueos	Send + esperar respuesta
CALL-TIMEOUT	$\mathcal{O}(1)$	$\leq N$ ticks	Plazo garantizado
CH-SEND	$\mathcal{O}(1)$	$\mathcal{O}(1)$ + bloqueo	Buffer no lleno
CH-RECV	$\mathcal{O}(1)$	$\mathcal{O}(1)$ + bloqueo	Buffer no vacío
Cap transfer	$\mathcal{O}(1)$	—	Verificación + inserción

Capítulo 4

Planificación

El planificador de SOTOS gestiona la multiplexación temporal del (o los) procesador(es) entre hilos concurrentes. Dado que todos los servicios del sistema operativo se ejecutan como procesos de usuario, el planificador es el árbitro final de los recursos de cómputo.

4.1. Multi-Level Queue (MLQ)

4.1.1. Estructura de Colas

Definición 4.1 (Cola Multinivel)**.** El planificador mantiene, por cada CPU, un arreglo de $P = 4$ colas FIFO independientes indexadas por *clase de prioridad*:

$$Q_{\text{cpu}}[k], \quad k \in \{0, 1, 2, 3\}$$

donde las clases son:

k	**Clase**	**Rango de prioridad**
0	Tiempo real	$[0, 63]$
1	Alta	$[64, 127]$
2	Normal	$[128, 191]$
3	Baja	$[192, 255]$

La función de mapeo es:

$$\text{class}(p) = \text{mín}\left(\left\lfloor \frac{p}{64} \right\rfloor, 3\right)$$

4.1.2. Algoritmo de Despacho

Algoritmo 9: DEQUEUE-LOCAL(Q_{cpu}): Despachar el siguiente hilo de la CPU local.

Input: Colas $Q_{\text{cpu}}[0.,3]$ de la CPU actual
Output: Índice de hilo o $\perp$

```
for k ← 0 to 3 do
    if Q_cpu[k] ≠ ∅ then
        return Q_cpu[k].PopFront();
return ⊥;
```

El algoritmo siempre despacha del nivel de mayor prioridad, lo que proporciona preemption estricta: un hilo de tiempo real siempre se ejecuta antes que uno normal.

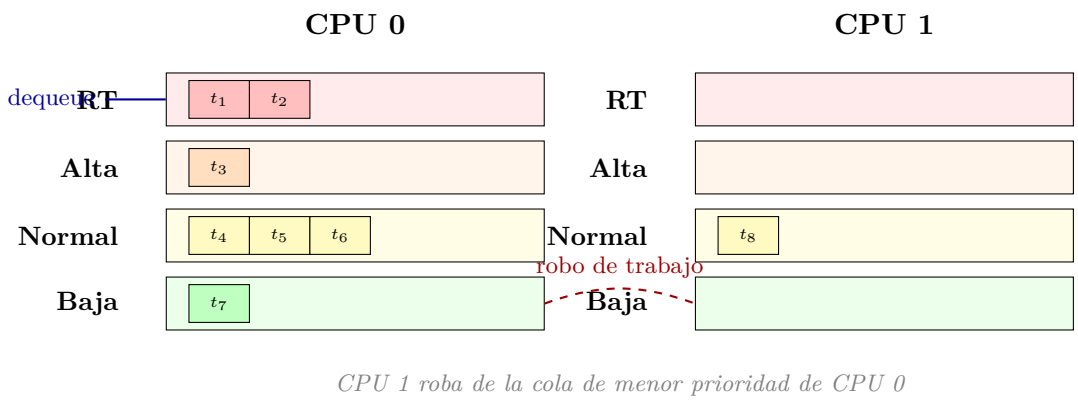

Figura 4.1: Colas multinivel por CPU con robo de trabajo. Cada CPU despacha primero de su cola de mayor prioridad (RT). Cuando la CPU queda ociosa, roba del extremo de menor prioridad de otra CPU.

4.2. Robo de Trabajo (Work Stealing)

4.2.1. Algoritmo

Cuando una CPU agota su cola local, intenta "robar" trabajo de otras CPUs.

Algoritmo 10: DEQUEUE-ANY(my_cpu): Despachar con robo de trabajo.

Input: Índice de la CPU actual my_cpu
Output: Índice de hilo o $\perp$

```
// Intentar cola local primero
t ← DEQUEUE-LOCAL(Q_my_cpu);
if t ≠ ⊥ then return t;
// Robar de otras CPUs (round-robin)
for offset ← 1 to C − 1 do
    victim ← (my_cpu + offset) mód C;
    if TRYLOCK(Q_victim) tiene éxito then
        t ← Q_victim.POPBACKLOWEST();
        UNLOCK(Q_victim);
        if t ≠ ⊥ then return t;
return ⊥;                                   // Todas las CPUs ociosas
```

Aspectos clave del diseño:

1. **Selección de víctima**: Round-robin simple (my_cpu + 1, +2, ...), lo que distribuye la carga de robo uniformemente.

2. **Selección de tarea**: Se roba del *extremo trasero* de la cola de *menor prioridad* (POPBACKLOWEST). Esto tiene dos beneficios: (a) las tareas de alta prioridad de la víctima no se perturban; y (b) las tareas antiguas ("frías" en cache de la víctima) tienen menor costo de migración.

3. **Try-lock**: Se usa TRYLOCK (no bloqueo) para evitar inversión de prioridad y deadlock si la víctima también está operando sobre su cola.

4.2.2. Análisis de Rendimiento

Teorema 4.1 (Cota de Makespan con Robo de Trabajo)**.** *Sea un programa con trabajo total T_1 (suma de tiempos de todas las tareas) y profundidad T_∞ (longitud del camino crítico). Con P procesadores y robo de trabajo aleatorizado, el makespan esperado satisface:*

$$\mathbb{E}[T_P] = \mathcal{O}\left(\frac{T_1}{P} + T_\infty\right).$$

Referencia de la demostración. Este resultado fue demostrado por Blumofe y Leiserson [1] para el planificador de Cilk-5. La prueba se basa en una *analogía de bola-en-urna*:

1. En cada paso temporal, si una CPU tiene trabajo local, lo ejecuta (paso productivo).
2. Si está ociosa, roba de una CPU elegida uniformemente al azar.
3. Un argumento de potencial muestra que el número esperado de robos es $\mathcal{O}(P \cdot T_\infty)$.
4. Cada paso es o productivo o un intento de robo. Los pasos productivos suman T_1 (el trabajo total). Los robos suman $\mathcal{O}(P \cdot T_\infty)$ en expectativa.
5. El makespan total es $(T_1 + \mathcal{O}(P \cdot T_\infty))/P = \mathcal{O}(T_1/P + T_\infty)$.

SOTOS usa selección round-robin en lugar de aleatoria, lo que provee propiedades similares en la práctica para P pequeño (típicamente ≤ 16 cores). □

Observación 4.1. La cota $\mathcal{O}(T_1/P + T_\infty)$ es óptima en el sentido de que:

- T_1/P es el tiempo mínimo con paralelismo perfecto (trabajo dividido equitativamente).
- T_∞ es el tiempo mínimo con infinitos procesadores (solo el camino crítico).

El speedup lineal se alcanza cuando $T_1/T_\infty \gg P$ (el *paralelismo* T_1/T_∞ excede el número de procesadores).

4.3. Cambio de Contexto

4.3.1. Salvar y Restaurar Registros

El cambio de contexto es una operación a nivel de ensamblador que salva los registros callee-saved del hilo actual y restaura los del hilo destino.

Definición 4.2 (TrapFrame)**.** El *TrapFrame* almacenado en la pila del kernel durante un cambio de contexto contiene los 6 registros callee-saved del ABI System V x86-64:

$$\text{TrapFrame} = (\texttt{r15}, \texttt{r14}, \texttt{r13}, \texttt{r12}, \texttt{rbx}, \texttt{rbp})$$

más la dirección de retorno implícita en el tope de la pila.

```
context_switch:
    push rbp
    push rbx
    push r12
    push r13
    push r14
    push r15
    mov [rdi], rsp       ; salvar RSP del hilo actual
    mov rsp, rsi         ; cargar RSP del hilo destino
    pop r15
    pop r14
    pop r13
    pop r12
    pop rbx
    pop rbp
    ret                   ; saltar a la continuaci n del nuevo hilo
```

Listing 4.1: Rutina de cambio de contexto (`context_switch`). RDI = puntero a RSP antiguo; RSI = RSP nuevo.

Proposición 4.2 (Costo del Cambio de Contexto)**.** *El cambio de contexto de SOTOS requiere exactamente:*

- *6 instrucciones* `push` *+ 1* `mov [mem], reg`*: salvar contexto (7 uops).*
- *1* `mov reg, mem` *+ 6 instrucciones* `pop` *+ 1* `ret`*: restaurar contexto (8 uops).*
- *Total: 15 uops ≈ 5–10 nanosegundos en hardware moderno.*

4.3.2. Transición a Modo Usuario: SYSCALL/SYSRET

Para hilos de usuario, el cambio de contexto termina con una transición de Ring 0 a Ring 3 mediante la instrucción `sysretq`.

Definición 4.3 (Configuración de MSRs para SYSCALL/SYSRET)**.** Las siguientes Model-Specific Registers (MSRs) se configuran durante la inicialización:

- `IA32_STAR` (0xC0000081): Segmentos para SYSCALL (kernel CS/SS) y SYSRET (user CS/SS).
- `IA32_LSTAR` (0xC0000082): Dirección del punto de entrada del kernel para SYSCALL.
- `IA32_FMASK` (0xC0000084): Máscara de RFLAGS aplicada en SYSCALL (deshabilita interrupciones durante la transición).

El flujo completo de un `sysretq` es:

1. Cargar CR3 del hilo destino (cambio de espacio de direcciones).
2. Cargar `rcx` ← RIP del usuario.
3. Cargar `r11` ← `0x202` (RFLAGS con IF=1, habilita interrupciones).
4. Cargar `rsp` ← RSP del usuario.
5. Ejecutar `swapgs` (intercambiar GS base kernel/usuario).
6. Ejecutar `sysretq` (transición a Ring 3).

4.3.3. Restauración de TLS y Estado Extendido

Además de los registros de propósito general, el cambio de contexto restaura:

- `IA32_FS_BASE` (MSR 0xC0000100): Base del segmento FS, usado por Thread-Local Storage (TLS). Cada hilo tiene su propio valor, restaurado via `wrmsr`.
- `IA32_KERNEL_GS_BASE` (MSR 0xC0000102): Valor de GS del usuario (guardado mientras el kernel usa GS para `PerCpu`).
- Estado FPU/SSE: 512 bytes salvados con `fxsave64` y restaurados con `fxrstor64`, necesarios porque instrucciones SIMD como `movaps` (usadas por musl/Rust para `memcpy`) corrompen registros XMM entre cambios de contexto.

4.4. Dominios de Planificación

Definición 4.4 (Dominio de Planificación)**.** Un *dominio de planificación* $D = (Q_D, P_D, M_D)$ donde:

- $Q_D \in \mathbb{N}$: quantum de ticks por período.
- $P_D \in \mathbb{N}$: longitud del período en ticks.
- $M_D \subseteq \mathbb{T}$: conjunto de hilos miembros ($|M_D| \leq 64$).

El dominio mantiene un contador consumed $\leq Q_D$ de ticks consumidos en el período actual. Cuando consumed $= Q_D$, el dominio entra en estado Depleted y todos sus hilos son suspendidos hasta el siguiente refill (cuando global_ticks $\geq$ next_refill).

Teorema 4.3 (Aislamiento Temporal entre Dominios)**.** *Sea D_1, D_2 dos dominios con cuotas Q_1, Q_2 y período P. Entonces en cualquier ventana de P ticks, los hilos de D_i consumen a lo sumo Q_i ticks de CPU.*

Demostración. El planificador incrementa D.consumed en cada tick donde un hilo de D se está ejecutando. Cuando D.consumed $= Q_D$, todos los hilos de D son suspendidos (movidos a D.suspended) y no son elegibles para despacho (la función Dequeue-Non-Depleted los filtra). Por lo tanto, en cada período de P ticks, ningún hilo de D puede ejecutar más de Q_D ticks.

La restauración ocurre cuando global_ticks $\geq$ next_refill: D.consumed $\leftarrow 0$, D.state $\leftarrow$ Active, D.next_refill $\leftarrow$ next_refill $+ P$, y todos los hilos suspendidos son re-encolados. □

4.5. Límites de Recursos por Hilo

Cada TCB mantiene contadores de consumo individual:

Definición 4.5 (Límites de Recursos)**.** Para cada hilo t:

- t.`cpu_ticks`: ticks de CPU consumidos (acumulativo).
- t.`cpu_tick_limit`: límite máximo (0 = ilimitado).
- t.`mem_pages`: páginas de memoria asignadas.
- t.`mem_page_limit`: límite máximo (0 = ilimitado).

Teorema 4.4 (El Agotamiento de Recursos Termina al Hilo Infractor)**.** *Si un hilo t alcanza $t.\texttt{cpu_ticks} \geq t.\texttt{cpu_tick_limit} > 0$, el planificador lo marca como* Dead*. Ningún otro hilo se ve afectado.*

Demostración. En la función Tick del planificador (invocada en cada interrupción de timer):

1. Se incrementa $t.\texttt{cpu_ticks}$ para el hilo actual.

2. Se verifica: si $t.\texttt{cpu_tick_limit} > 0$ y $t.\texttt{cpu_ticks} \geq t.\texttt{cpu_tick_limit}$, entonces $t.\texttt{state} \leftarrow$ Dead.

3. El hilo muerto nunca es re-encolado (la función Enqueue no acepta hilos en estado Dead).

Los demás hilos no son afectados porque: (a) cada hilo tiene contadores independientes; y (b) el planificador preemptivo cambia al siguiente hilo listo inmediatamente después de marcar al infractor como muerto. □

4.6. Asignación de Marcos Físicos

Aunque la gestión de memoria virtual (paginación) se delega al servidor VMM en espacio de usuario, la asignación de *marcos físicos* — las unidades atómicas de memoria física de 4 KiB — es una primitiva del kernel.

4.6.1. Asignador de Bitmap

Definición 4.6 (Bitmap de Marcos)**.** El asignador mantiene un arreglo de bits $B[0..N)$ donde N es el número total de marcos físicos en el sistema:

$$B[i] = \begin{cases} 0 & \text{si el marco } i \text{ está libre} \\ 1 & \text{si el marco } i \text{ está asignado} \end{cases}$$

Un marco tiene dirección física $\text{addr}(i) = i \times 4096$.

Algoritmo 11: FRAME-ALLOC(B): Asignar un marco físico libre.

```
Input: Bitmap B con hint h
Output: Marco físico o ⊥
// Dos pasadas: hint..N, luego 0..hint
for pass ← 0 to 1 do
    (s, e) ← if pass = 0 then (h, N) else (0, h);
    i ← s;
    while i < e do
        byte ← i/8;
        if B[byte] = 0xFF then
            i ← (byte + 1) × 8;                    // Skip: byte completo ocupado
            continue;
        for b ← i mód 8 to 7 do
            j ← byte × 8 + b;
            if j ≥ e then
                break
            if B[j] = 0 then
                B[j] ← 1; h ← j + 1;
                return PhysFrame(j × 4096);
        i ← (byte + 1) × 8;
return ⊥;                                          // Sin marcos libres
```

Proposición 4.5 (Complejidad del Asignador de Marcos)**.**

1. *FRAME-ALLOC: $\mathcal{O}(N/8)$ peor caso (escaneo completo del bitmap a nivel de byte), $\mathcal{O}(1)$ caso típico (el hint apunta a un marco libre).*

2. *FRAME-FREE: $\mathcal{O}(1)$ (acceso directo por índice).*

3. *FRAME-ALLOC-CONTIGUOUS(k): $\mathcal{O}(N \cdot k)$ peor caso (buscar secuencia de k marcos libres consecutivos).*

Demostración.

1. La optimización de *byte skip* ($B[\text{byte}] = \texttt{0xFF}$) reduce el escaneo a $N/8$ comparaciones en el peor caso. El hint h memoriza la posición del último marco asignado, proporcionando localidad temporal: si hay marcos libres cerca del último asignado, la búsqueda es $\mathcal{O}(1)$.

2. FRAME-FREE calcula $\text{byte} = i/8$, $\text{bit} = i \text{ mód } 8$ y limpia $B[\text{byte}][\text{bit}]$: una división, un AND y una escritura.

3. La búsqueda contigua verifica cada posición candidata i y luego escanea k bits consecutivos. En el peor caso (bitmap casi lleno), cada candidato falla y se necesitan $N - k + 1$ intentos de k verificaciones cada uno.

□

4.6.2. Inicialización desde el Mapa de Memoria

El asignador se inicializa a partir del mapa de memoria proporcionado por el cargador Limine:

1. Se determina la dirección física más alta de RAM utilizable para dimensionar el bitmap: $N = \lceil \text{max_addr}/4096 \rceil$.
2. Se asigna el bitmap ($\lceil N/8 \rceil$ bytes) dentro de una región utilizable.
3. Se marca *todo* como ocupado (`memset(0xFF)`).
4. Se recorren las regiones `USABLE` del mapa de memoria, liberando los marcos correspondientes (excepto los que contienen el bitmap mismo).

Invariante 4.1 (Consistencia del Bitmap). En todo momento, $B[i] = 1$ si y solo si el marco i ha sido asignado y no liberado. Un marco asignado por Frame-Alloc no puede ser asignado nuevamente hasta que sea liberado por Frame-Free.

Demostración. Por inducción sobre la secuencia de operaciones:

- **Base**: Después de la inicialización, todos los marcos de regiones no utilizables tienen $B[i] = 1$ (marcados como ocupados). Los marcos de regiones utilizables tienen $B[i] = 0$ (libres). El bitmap del asignador mismo está marcado como ocupado.
- **Paso (Alloc)**: Frame-Alloc solo selecciona i con $B[i] = 0$, luego establece $B[i] = 1$. No se cambia ningún otro bit.
- **Paso (Free)**: Frame-Free verifica que $B[i] = 1$ (assertion de doble liberación) y establece $B[i] = 0$. No se cambia ningún otro bit.

Todas las operaciones están protegidas por un spinlock global, por lo que no hay carreras de datos. □

4.7. Resumen del Capítulo

Este capítulo ha presentado los cuatro mecanismos centrales del planificador de sotOS:

1. **MLQ por CPU**: Cuatro niveles de prioridad con despacho estricto por prioridad, proporcionando determinismo para hilos de tiempo real.
2. **Robo de trabajo**: Balanceo de carga automático con cota teórica $\mathcal{O}(T_1/P + T_\infty)$.
3. **Dominios de planificación**: Aislamiento temporal con cuotas periódicas, garantizando que un dominio no puede monopolizar la CPU.
4. **Límites de recursos**: Protección individual por hilo contra consumo excesivo de CPU y memoria.

Junto con el asignador de marcos físicos, estos mecanismos completan las cinco primitivas del kernel descritas en el chapter 1.

Parte II

Espacio de Usuario

Capítulo 5

Gestión de Memoria

La gestión de memoria en SOTOS se divide en dos niveles:

1. **Asignador de marcos físicos** (en el núcleo): administra las páginas de 4 KiB de RAM física mediante un bitmap.

2. **Tablas de páginas y memoria virtual** (en espacio de usuario): el servidor VMM maneja los fallos de página, la paginación bajo demanda y el Copy-on-Write.

Esta separación de competencias es la decisión arquitectónica central del micronúcleo: el kernel nunca interpreta direcciones virtuales de usuario, delegando toda la política de memoria virtual al servidor VMM.

5.1. Asignador de Marcos Físicos

5.1.1. Estructura de Datos

El asignador utiliza un bitmap global protegido por un `spin::Mutex`. Cada bit representa un marco de 4 KiB: bit 0 indica libre, bit 1 indica ocupado.

Definición 5.1 (Bitmap de marcos)**.** Sea $N = \lceil max_addr/4096 \rceil$ el número total de marcos rastreables. El bitmap $B[0..N-1]$ es un arreglo de $\lceil N/8 \rceil$ bytes, donde el bit j del byte i corresponde al marco $8i + j$.

Además del bitmap, el asignador mantiene:

- `total_frames`: número total de marcos rastreados.

- `free_frames`: contador de marcos libres (decrementado en alloc, incrementado en free).

- `search_hint`: índice del último marco asignado + 1, punto de inicio para la siguiente búsqueda.

5.1.2. Algoritmo de Asignación

Algoritmo 12: FRAME-ALLOC: Asignación de un marco físico

Input: Bitmap $B[0..N-1]$, puntero de búsqueda `hint`
Output: Dirección física del marco asignado, o error

```
for pass ← 0 to 1 do
    (s, e) ← { (hint, N)   si pass = 0
             { (0, hint)   si pass = 1
    i ← s
    while i < e do
        byte ← ⌊i/8⌋
        if B[byte] = 0xFF then
            i ← (byte + 1) × 8 ;                    // saltar byte completo
            continuar
        for b ← (i mód 8) to 7 do
            idx ← byte × 8 + b
            if idx ≥ e then
                salir del bucle
            if bit(B, idx) = 0 then
                set_bit(B, idx)
                free_frames ← free_frames − 1
                hint ← idx + 1
                return idx × 4096
        i ← (byte + 1) × 8
return None ;                                       // sin marcos libres
```

Teorema 5.1 (Complejidad del asignador bitmap)**.** *Sea N el número total de marcos y F el número de marcos libres.*

1. *En el peor caso, FRAME-ALLOC es $\mathcal{O}(N/8)$ (escaneo completo, byte a byte).*
2. *En el caso amortizado con puntero de búsqueda (`hint`), si las liberaciones son infrecuentes respecto a las asignaciones, FRAME-ALLOC es $\mathcal{O}(1)$ amortizado.*

Demostración. 1. **Peor caso.** El bucle exterior itera sobre como máximo $\lceil N/8 \rceil$ bytes (dos pasadas concatenadas cubren todos los bytes una vez). Cada byte se examina en tiempo $\mathcal{O}(1)$: si vale `0xFF` se salta; si no, se busca el primer bit libre dentro de 8 posiciones. Total: $\mathcal{O}(N/8)$.

2. **Amortizado.** Consideremos una secuencia de k asignaciones consecutivas sin liberaciones. Después de cada asignación, `hint` se adelanta al siguiente índice. La siguiente asignación comienza en `hint` y encuentra un marco libre en $\mathcal{O}(1)$ si los marcos están contiguos (caso común en un bitmap recién inicializado). La suma total de trabajo para k asignaciones es a lo sumo $N/8$ (un recorrido completo), dando $\mathcal{O}(N/(8k))$ amortizado por operación. Cuando $k \geq N/8$, el costo amortizado es $\mathcal{O}(1)$. □

5.1.3. Asignación Contigua para DMA

Ciertos dispositivos (virtqueues de virtio, PRP lists de NVMe) requieren marcos físicamente contiguos. La función `alloc_contiguous(count)` extiende el algoritmo anterior:

Algoritmo 13: FRAME-ALLOC-CONTIGUOUS: Asignación de k marcos contiguos

```
Input: Bitmap B[0..N − 1], cantidad k
Output: Dirección física base, o error
run ← 0
run_start ← 0
for i ← 0 to N − 1 do
    if bit(B, i) = 0 then
        if run = 0 then
            run_start ← i
        run ← run + 1
        if run = k then
            for j ← run_start to run_start + k − 1 do
                set_bit(B, j)
            free_frames ← free_frames − k
            return run_start × 4096
    else
        run ← 0
return None
```

La complejidad es $\mathcal{O}(N)$ en el peor caso (escaneo lineal completo).

5.1.4. Tabla de Refcount para CoW

Para soportar Copy-on-Write, el módulo `mm` mantiene una tabla global de contadores de referencia:

$$\texttt{FRAME_REFCOUNT}[i] \in \{0, 1, 2, \ldots, 65535\}, \quad i = \texttt{phys} \gg 12$$

- `rc` $= 0$: marco sin rastreo (propietario único, caso normal).
- `rc` $= 1$: compartido pero con una sola referencia restante (puede hacerse escribible sin copia).
- `rc` ≥ 2: compartido, se requiere copia CoW al escribir.

La tabla soporta hasta 524 288 marcos (= 2 GiB de RAM), usando 1 MiB de memoria estática (`[u16; 524288]`).

5.2. Tablas de Páginas: x86-64 de Cuatro Niveles

5.2.1. Estructura de Traducción

La arquitectura x86-64 utiliza tablas de páginas de cuatro niveles para traducir direcciones virtuales de 48 bits a direcciones físicas:

Cada entrada de tabla de páginas (PTE) tiene 64 bits con la siguiente estructura:

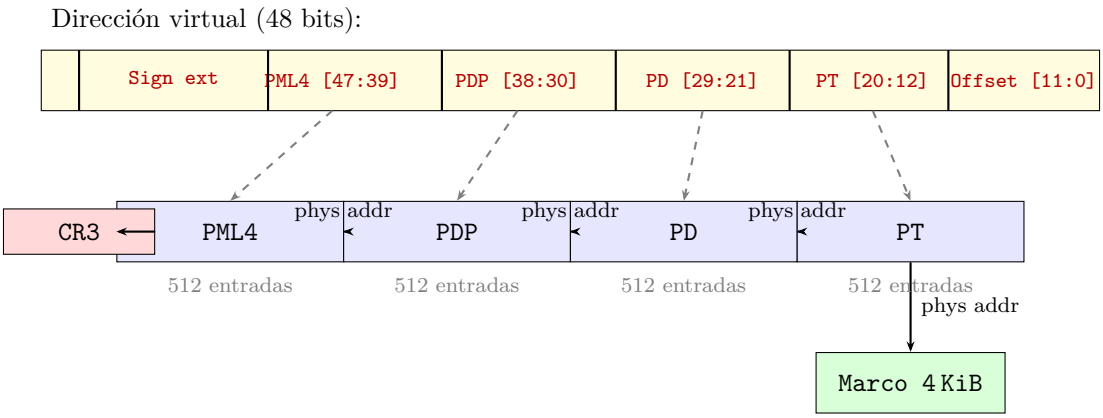

Figura 5.1: Traducción de direcciones virtuales x86-64 con tablas de páginas de cuatro niveles. Cada nivel tiene 512 entradas de 8 bytes (4 KiB por tabla). Los bits de índice de la dirección virtual seleccionan la entrada en cada nivel.

Bit(s)	Nombre	Descripción
0	`PRESENT`	Entrada válida (si = 0, acceso genera #PF)
1	`WRITABLE`	Página escribible (si = 0, escritura genera #PF)
2	`USER`	Accesible desde Ring 3
3	`WRITE_THROUGH`	Write-through caching
4	`CACHE_DISABLE`	Deshabilitar caché (para MMIO)
12–51	Dirección física	Bits [51:12] de la dirección física del marco
63	`NO_EXECUTE`	Ejecución prohibida (requiere NXE en IA32_EFER)

5.2.2. Extracción de Índices

Dado un virtual address v, los índices para cada nivel se calculan como:

$$\text{pml4_index}(v) = (v \gg 39)\,\&\,\texttt{0x1FF} \tag{5.1}$$

$$\text{pdp_index}(v) = (v \gg 30)\,\&\,\texttt{0x1FF} \tag{5.2}$$

$$\text{pd_index}(v) = (v \gg 21)\,\&\,\texttt{0x1FF} \tag{5.3}$$

$$\text{pt_index}(v) = (v \gg 12)\,\&\,\texttt{0x1FF} \tag{5.4}$$

5.2.3. Algoritmos de Tabla de Páginas

Algoritmo 14: MAP-PAGE: Mapear una página virtual a un marco físico

Input: Espacio de direcciones AS (PML4 físico), dirección virtual v, dirección física p, flags f

```
hhdm ← offset del Higher Half Direct Map
table_flags ← PRESENT | WRITABLE | USER
// PML4 → PDP
pml4 ← (AS.pml4_phys + hhdm) como puntero
pdp_phys ← ENSURE-TABLE(pml4[pml4_index(v)], table_flags)
// PDP → PD
pdp ← (pdp_phys + hhdm) como puntero
pd_phys ← ENSURE-TABLE(pdp[pdp_index(v)], table_flags)
// PD → PT
pd ← (pd_phys + hhdm) como puntero
pt_phys ← ENSURE-TABLE(pd[pd_index(v)], table_flags)
// Escribir entrada hoja
pt ← (pt_phys + hhdm) como puntero
pt[pt_index(v)] ← (p & !0xFFF) | f
```

Algoritmo 15: ENSURE-TABLE: Garantizar existencia de tabla intermedia

Input: Referencia a entrada e, flags f

Output: Dirección física de la tabla apuntada

```
if e & PRESENT ≠ 0 then
    return e & 0x000F_FFFF_FFFF_F000 ;          // extraer phys
else
    frame ← ALLOC-TABLE-FRAME() ;               // 4 KiB, puesto a cero
    e ← frame | f
    return frame
```

Algoritmo 16: LOOKUP-PHYS: Traducir dirección virtual a física

Input: Espacio de direcciones AS, dirección virtual v
Output: Dirección física, o `None`

```
hhdm ← offset HHDM
// PML4
pml4e ← *(AS.pml4_phys + hhdm)[pml4_index(v)]
if pml4e&PRESENT = 0 then
    return None
pdp_phys ← pml4e & 0x000F_FFFF_FFFF_F000
// PDP
pdpe ← *(pdp_phys + hhdm)[pdp_index(v)]
if pdpe&PRESENT = 0 then
    return None
pd_phys ← pdpe & 0x000F_FFFF_FFFF_F000
// PD
pde ← *(pd_phys + hhdm)[pd_index(v)]
if pde&PRESENT = 0 then
    return None
pt_phys ← pde & 0x000F_FFFF_FFFF_F000
// PT (hoja)
pte ← *(pt_phys + hhdm)[pt_index(v)]
if pte&PRESENT = 0 then
    return None
return pte & 0x000F_FFFF_FFFF_F000
```

Algoritmo 17: UNMAP-PAGE: Desmapear una página virtual

Input: Espacio de direcciones AS, dirección virtual v
Output: `Some(phys)` si la página existía, `None` si no

```
Recorrer los cuatro niveles como en LOOKUP-PHYS
if algún nivel intermedio no está presente then
    return None
pte ← referencia mutable a la entrada hoja
if pte&PRESENT = 0 then
    return None
phys ← pte & 0x000F_FFFF_FFFF_F000
pte ← 0 ;                                   // borrar entrada
return Some(phys)
```

5.3. Copy-on-Write (CoW)

El mecanismo de Copy-on-Write permite que `fork()` sea eficiente: en lugar de copiar toda la memoria del proceso padre, ambos procesos comparten las mismas páginas físicas marcadas como solo lectura. Cuando cualquiera de los dos escribe en una página compartida, se genera un fallo de página que el VMM resuelve copiando la página.

5.3.1. Algoritmo de Clonación CoW

Algoritmo 18: CLONE-COW: Clonar un espacio de direcciones con Copy-on-Write

```
Input: Espacio de direcciones padre P
Output: Nuevo espacio de direcciones hijo C
C.pml4_phys ← ALLOC-TABLE-FRAME()
// Copiar mitad del kernel (entradas 256..511)
for i ← 256 to 511 do
    C.pml4[i] ← P.pml4[i]
// Clonar mitad de usuario (entradas 0..255)
for i4 ← 0 to 255 do
    if P.pml4[i4] no está presente then
        continuar
    Asignar nueva PDP para C
    for i3 ← 0 to 511 do
        if PDP padre [i3] no está presente then
            continuar
        Asignar nuevo PD para C
        for i2 ← 0 to 511 do
            if PD padre [i2] no está presente then
                continuar
            Asignar nueva PT para C
            for i1 ← 0 to 511 do
                pte ← PT padre [i1]
                if pte no está presente then
                    continuar
                phys ← pte & PHYS_MASK
                // Hijo: PTE sin bit WRITABLE
                C.PT[i1] ← pte & ¬WRITABLE
                // Padre: marcar read-only (CoW simétrico)
                if pte era writable ∧ no es MMIO ∧ no es región de servicio then
                    P.PT[i1] ← pte & ¬WRITABLE
                // Refcount: 0→2 (primer compartimiento), >0→+1
                if REFCOUNT[phys ≫ 12] = 0 then
                    REFCOUNT[phys ≫ 12] ← 2
                else
                    REFCOUNT[phys ≫ 12] += 1
// Flush TLB del padre (PTEs cambiaron a read-only)
FLUSH-TLB()
return C
```

Observación 5.1. El CoW es *simétrico*: ambos, padre e hijo, pierden el permiso de escritura. Esto garantiza que el *primer* escritor (sea padre o hijo) pague el costo de la copia, no solo el hijo. Las páginas UC/MMIO y las regiones de servicio del proceso `init` se excluyen para no romper DMA ni

el handler de IPC.

5.3.2. Manejo de Fallos CoW en el VMM

Algoritmo 19: CoW-Fault-Handler: Resolución de fallo de escritura CoW

Input: Fallo de página con código c y dirección v, espacio de direcciones AS

```
// Detectar fallo CoW: escritura (bit 1) a página presente (bit 0)
if (c & 0x03) ≠ 0x03 then
    no es CoW, manejar como fallo normal
new_frame ← Frame-Alloc()
// Copiar 4 KiB del marco original al nuevo (vía HHDM en kernel)
Frame-Copy(new_frame, AS, v)
// Reemplazar el mapeo: desmapear viejo, mapear nuevo como WRITABLE
Unmap-From(AS, v)
Map-Into(AS, v, new_frame, WRITABLE)
Thread-Resume(tid)
```

Teorema 5.2 (CoW preserva aislamiento de memoria)**.** *Después de una operación Clone-CoW, toda escritura del padre o del hijo a una página compartida resulta en una copia privada. Formalmente: sean M_P y M_C las vistas de memoria del padre y del hijo respectivamente. Para toda dirección virtual v en el espacio compartido, después de una escritura $M_P[v] \leftarrow x$ o $M_C[v] \leftarrow x$, los marcos físicos subyacentes de $M_P[v]$ y $M_C[v]$ son distintos.*

Demostración. Sea v una dirección virtual compartida tras Clone-CoW. Sea f_0 el marco físico original.

Invariante: Tras Clone-CoW, la PTE de v en *ambos* espacios de direcciones tiene el bit `WRITABLE` desactivado.

Caso 1: El padre escribe en v. La escritura genera un #PF con código c donde c `&` `0x03` $=$ `0x03` (escritura a página presente). El VMM ejecuta CoW-Fault-Handler:

1. Asigna un nuevo marco f_P.
2. Copia el contenido de f_0 a f_P (4 KiB).
3. Desmapea $v \to f_0$ en el padre y mapea $v \to f_P$ con `WRITABLE`.

Ahora $M_P[v]$ reside en f_P y $M_C[v]$ sigue en f_0. Son marcos distintos: $f_P \neq f_0$ porque Frame-Alloc nunca retorna un marco ya asignado (por el invariante 4.1).

Caso 2: El hijo escribe en v. Análogo al Caso 1, con los roles invertidos. El hijo obtiene un nuevo marco $f_C \neq f_0$.

Caso 3: Ambos escriben. Después de la primera escritura (digamos del padre), tenemos $M_P[v] \to f_P$ y $M_C[v] \to f_0$. Cuando el hijo escribe, si f_0 aún tiene `REFCOUNT` ≥ 2, se genera otro fallo CoW y el hijo obtiene $f_C \neq f_0$. En total, f_P, f_C, y f_0 (si aún existe) son marcos distintos.

En todos los casos, la escritura resulta en marcos físicos independientes, lo que preserva el aislamiento. □

Teorema 5.3 (Cota de memoria para CoW)**.** *Sea n el número de páginas del proceso original. Después de un `fork()` con CoW:*

1. *La memoria total inmediatamente después del fork es n páginas (compartidas) más $\mathcal{O}(n/512)$ páginas para las tablas de páginas del hijo (una PT por cada 512 entradas presentes).*

2. *En el peor caso, si ambos procesos escriben en todas las páginas, la memoria total es* $\leq 2n$ *páginas de datos más tablas.*

Demostración. 1. Inmediatamente tras Clone-CoW, no se duplica ninguna página de datos: los marcos físicos son compartidos. Solo se asignan nuevas tablas de páginas (PML4, PDP, PD, PT) para el hijo. En el peor caso, si las n páginas están dispersas en n PTs distintas, se necesitan n PTs + hasta $n/512$ PDs + hasta $n/512^2$ PDPs + 1 PML4. Esto es $\mathcal{O}(n/512)$ en el caso típico donde las páginas están agrupadas.

2. Cada escritura CoW duplica exactamente una página (4 KiB). Si el padre escribe en las n páginas, genera n copias. El hijo conserva las n originales. Total: $2n$ páginas de datos. Análogamente si solo el hijo escribe, o si ambos escriben (máximo $2n$ en cualquier caso). □

5.4. Enforcement W^X via mprotect

sotOS aplica la política Write XOR Execute (W^X) a nivel de PTE: una página no puede tener simultáneamente los bits `WRITABLE` y la ausencia de `NO_EXECUTE`. La syscall `SYS_PROTECT` (134) permite cambiar los permisos de una página existente con esta restricción.

Definición 5.2 (Propiedad W^X)**.** Para toda PTE e con $e \,\&\, \texttt{PRESENT} \neq 0$ en un espacio de direcciones sin relajación:

$$(e \,\&\, \texttt{WRITABLE} \neq 0) \implies (e \,\&\, \texttt{NO_EXECUTE} \neq 0)$$

Equivalentemente, si una página es escribible, no es ejecutable, y viceversa.

Para binarios que requieren páginas RWX (como Wine/glibc con IAT fixups o JIT), se ofrece un mecanismo de relajación por espacio de direcciones mediante un bitmap atómico `WX_RELAXED`. Cuando está activado, el kernel no añade `NO_EXECUTE` automáticamente a páginas escribibles de ese AS.

Capítulo 6

LUCAS — Capa de Compatibilidad Linux

LUCAS (*Linux User Compatibility Abstraction Shim*) es la capa de espacio de usuario que permite ejecutar binarios de Linux sin modificación sobre SOTOS. Intercepta las llamadas al sistema Linux del proceso huésped, las traduce a operaciones sobre los servicios del micronúcleo, y devuelve resultados en el formato esperado por la ABI de Linux.

6.1. Arquitectura del Redireccionamiento de Syscalls

Cuando un proceso huésped ejecuta la instrucción `syscall`, el kernel de SOTOS detecta que el hilo tiene un *endpoint de redirección* configurado y, en lugar de manejar la syscall directamente, la entrega como un mensaje IPC al manejador LUCAS.

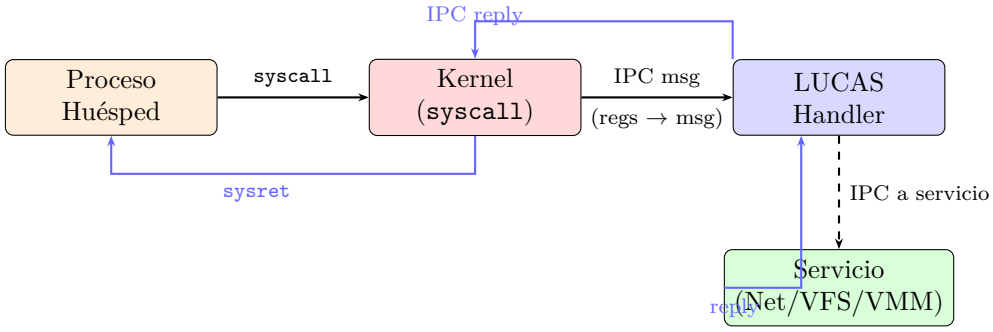

Figura 6.1: Flujo de una syscall Linux en SOTOS. El kernel redirige la syscall como mensaje IPC a LUCAS, que opcionalmente delega a un servicio del sistema, y devuelve el resultado al huésped vía IPC reply y `SYSRET`.

6.1.1. Recorrido Completo de una Syscall: `write(1, "Hola", 4)`

Para ilustrar el mecanismo, rastreamos una llamada `write(fd=1, buf="Hola", len=4)` paso a paso:

1. **Huésped**: El binario Linux coloca `RAX=1` (SYS_write), `RDI=1` (fd), `RSI=ptr` (buffer), `RDX=4` (longitud) y ejecuta `syscall`.

2. **Kernel (entrada)**: El handler de `SYSCALL` detecta que el hilo tiene `redirect_ep` $\neq$ `0`. Guarda los registros del huésped en un `TrapFrame` y construye un mensaje IPC con los valores de RAX, RDI, RSI, RDX, R10, R8, R9. Entrega el mensaje al endpoint de redirección (IPC call bloqueante).

3. **LUCAS (child_handler)**: Recibe el mensaje IPC. Extrae `syscall_nr = regs[0]` (= 1, SYS_WRITE). El manejador del módulo `fs.rs` lee 4 bytes desde la dirección `RSI` en el espacio de memoria del huésped (usando `SYS_GET_THREAD_REGS` para acceso remoto o memoria compartida). Si `fd=1` (stdout), escribe los bytes a la consola serial vía `sys::debug_print()`.

4. **LUCAS (reply)**: Construye el mensaje de respuesta con `regs[0] = 4` (bytes escritos), y usa la etiqueta `SIG_REDIRECT_TAG` para restaurar el RIP/RSP/RAX del huésped. Responde por IPC.

5. **Kernel (retorno)**: Recibe el reply, aplica el `SIG_REDIRECT_TAG`: establece el RIP, RSP y RAX del huésped según los registros del reply. Ejecuta `SYSRET` para devolver el control al huésped.

6. **Huésped**: Reanuda ejecución después del `syscall` con `RAX=4` (éxito, 4 bytes escritos).

6.2. Emulación de la ABI Linux: 127+ Syscalls

El manejador LUCAS implementa más de 127 llamadas al sistema Linux, organizadas en seis módulos:

6.3. Modelo de Procesos: Fork/Exec/Wait

LUCAS emula el modelo de procesos POSIX completo:

6.3.1. Grupos de Hilos (`CLONE_THREAD`)

Cada proceso tiene un *thread group ID* (TGID) y puede crear hilos adicionales mediante `clone()` con el flag `CLONE_THREAD`. Los hilos de un grupo comparten:

- **Tabla de descriptores de archivo**: via `CLONE_FILES`. Las tablas están en arreglos estáticos por grupo (`GRP_FDS`, `GRP_INITRD`, `GRP_VFS`).
- **Espacio de direcciones**: via `CLONE_VM`. El hilo hijo comparte el mismo CR3.
- **Manejadores de señal**: via `CLONE_SIGHAND`.

6.3.2. Fork con CoW

La llamada `fork()` invoca `SYS_AS_CLONE` (125) en el kernel, que ejecuta Clone-CoW (algorithm 18). El hijo se configura con un trampolín en el vDSO que establece `RAX=0` (retorno de fork) y restaura los registros callee-saved:

```
cow_fork_restore:
    xor eax, eax          ; fork() retorna 0 en el hijo
    pop rbx
    pop rbp
```

Cuadro 6.1: Módulos del manejador LUCAS y syscalls representativas.

Módulo	Archivo	Syscalls principales
Sistema de archivos	`fs.rs`	open, openat, read, write, writev, close, lseek, fstat, fstatat, getdents64, mkdir, unlink, rename, dup/dup2/dup3, pipe, fcntl, ioctl, mmap (file-backed), pread64, pwrite64, preadv, pwritev
Memoria	`mm.rs`	brk, mmap, munmap, mprotect, mremap, madvise, memfd_create
Red	`net.rs`	socket, connect, bind, sendto, recvfrom, recvmsg, readv, poll, epoll_create/ctl/wait, select, pselect6
Señales	`signal.rs`	rt_sigaction, rt_sigprocmask, rt_sigreturn, rt_sigpending, rt_sigtimedwait, rt_sigsuspend, sigaltstack, tgkill
Procesos	`task.rs`	fork, clone, execve, wait4, waitid, exit, exit_group, getpid, gettid, set_tid_address, sched_yield
Información	`info.rs`	uname, getuid/gid/euid/egid, clock_gettime, clock_getres, gettimeofday, getrandom, getrusage, prlimit64, sysinfo

```
5       pop r12
6       pop r13
7       pop r14
8       pop r15
9       ret                     ; retorna al punto de fork del padre
```

Listing 6.1: Trampolín CoW fork child restore (vDSO 0xB80740).

6.3.3. Entrega de Señales

Las señales se entregan en dos modos:

1. **Síncrono** (en frontera de syscall): Si hay una señal pendiente cuando LUCAS devuelve un reply, modifica el RIP del huésped para saltar al manejador de señal antes de reanudar la ejecución normal.

2. **Asíncrono** (vía interrupción de timer): El kernel detecta señales pendientes en la rutina de interrupción del LAPIC timer (cada ∼10 ms). Si el hilo tiene `pending_signals` $\neq$ `0` y un `signal_trampoline` registrado, el kernel:

   a) Guarda el `TrapFrame` actual del huésped en un `SignalFrame` de 176 bytes en la pila del huésped.

   b) Redirige RIP al trampolín de señales en el vDSO (0xB80700).

c) El trampolín invoca el handler registrado por `sigaction()`.

d) Al retornar, el handler ejecuta `rt_sigreturn` (syscall 15), que restaura el `TrapFrame` original desde el `SignalFrame`.

6.4. Puente al Sistema de Archivos Virtual

6.4.1. ObjectStore: Almacenamiento Transaccional

El sistema de archivos de SOTOS se basa en el `ObjectStore`, un almacén de objetos sobre disco con escritura adelantada (WAL) para consistencia ante fallos. Los detalles del layout en disco se describen en el section 7.4.

6.4.2. Resolución de Rutas con CWD

Cada proceso mantiene un directorio de trabajo actual (`CWD`) en el arreglo `GRP_CWD`. La función `resolve_with_cwd()` resuelve rutas relativas concatenándolas con el CWD:

- Si la ruta comienza con `/`, se usa tal cual (ruta absoluta).
- Si no, se concatena `CWD + /` ruta`.
- Se aplica a todas las syscalls de archivo: open, openat, stat, access, mkdir, unlink, rename.

6.4.3. Snapshots CoW

El `ObjectStore` soporta hasta 4 snapshots simultáneos. Cada snapshot captura el estado del directorio y bitmap en un momento dado, permitiendo lecturas consistentes mientras las escrituras continúan en la copia activa.

6.5. vDSO: Shared Library Forjado

El *Virtual Dynamic Shared Object* (vDSO) es una página de 4 KiB mapeada en el espacio de direcciones de cada proceso en `0xB80000`. Contiene un ELF mínimo pero completo (`ET_DYN`) que exporta funciones de tiempo accesibles sin transición a Ring 0.

La función principal exportada, `__vdso_clock_gettime`, usa `RDTSC` para devolver el tiempo sin transición al kernel:

```
__vdso_clock_gettime:
    rdtsc                       ; EDX:EAX = timestamp counter
    shl rdx, 32
    or  rax, rdx                ; RAX = TSC completo (64 bits)
    ; Dividir por 2 GHz para obtener segundos
    mov rcx, 2000000000
    xor edx, edx
    div rcx                     ; RAX = segundos, RDX = nanosegundos
        residuales
    mov [rsi],    rax           ; tv_sec
    ; Convertir residuo a nanosegundos
    imul rdx, rdx, 1000000000
    mov rcx2, 2000000000
    div rcx                     ; RDX = nanosegundos
```

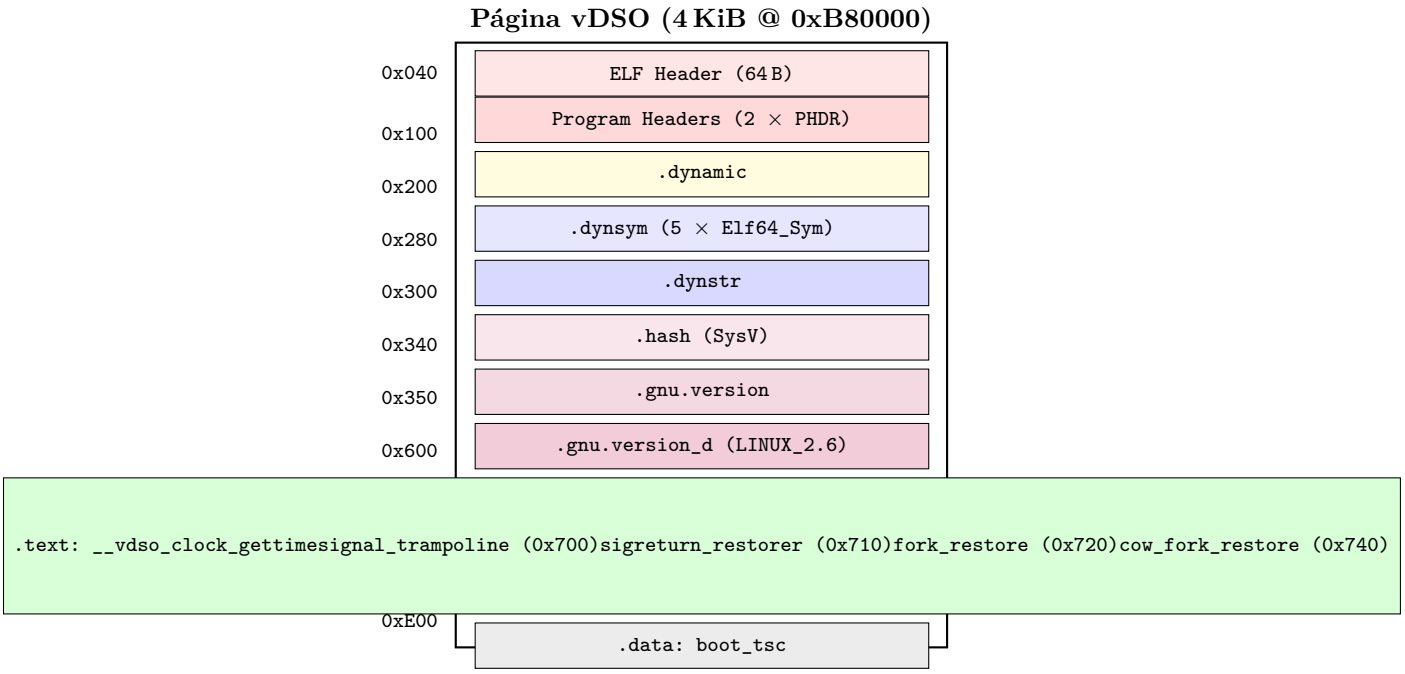

Figura 6.2: Layout de la página vDSO. El ELF es autocontenido en una única página de 4 KiB. Las secciones `.gnu.version` y `.gnu.version_d` con la cadena `"LINUX_2.6"` satisfacen la verificación de versión de musl y glibc.

```
14      mov [rsi+8],  rax          ; tv_nsec
15      xor eax, eax               ; retornar 0 (exito)
16      ret
```

Listing 6.2: Implementación de `__vdso_clock_gettime` en el vDSO.

El vDSO también contiene trampolines críticos para el funcionamiento del sistema:

Offset	Dirección	Función
0x700	0xB80700	Trampolín de señales
0x710	0xB80710	Restaurador `rt_sigreturn`
0x720	0xB80720	Fork restore (pop callee-saved + ret)
0x730	0xB80730	Exit stub (`exit_group`)
0x740	0xB80740	CoW fork restore (xor eax + pop + ret)
0x760	0xB80760	Fork TLS trampoline (arch_prctl + jump)
0x7C0	0xB807C0	Pre-TLS entry trampoline
0x7E0	0xB807E0	Yield loop (mantiene hilo vivo)

Capítulo 7

Servicios del Sistema

En la arquitectura de SOTOS, los servicios del sistema ejecutan en espacio de usuario como procesos independientes, cada uno con su propio espacio de direcciones (CR3). Se comunican con el kernel y entre sí exclusivamente mediante IPC (descrito en el chapter 3).

7.1. Servicio de Red (smoltcp)

El servicio de red implementa la pila TCP/IP completa usando la biblioteca `smoltcp`, proporcionando conectividad de red a los procesos huésped a través de una interfaz IPC de 13 comandos.

7.1.1. Interfaz de Comandos IPC

Cuadro 7.1: Comandos IPC del servicio de red.

Código	Comando	Descripción
1	`CMD_PING`	Enviar ICMP echo request y esperar reply
2	`CMD_DNS_QUERY`	Resolver nombre de dominio vía UDP (10.0.2.3)
3	`CMD_TCP_CONNECT`	Establecer conexión TCP (3-way handshake)
4	`CMD_TCP_SEND`	Enviar datos por conexión TCP
5	`CMD_TCP_RECV`	Recibir datos de conexión TCP
6	`CMD_TCP_CLOSE`	Cerrar conexión TCP (FIN)
7	`CMD_TRACEROUTE_HOP`	Enviar paquete con TTL específico
8	`CMD_UDP_BIND`	Vincular puerto UDP efímero
9	`CMD_UDP_SENDTO`	Enviar datagrama UDP
10	`CMD_UDP_RECV`	Recibir datagrama UDP
11	`CMD_TCP_STATUS`	Consultar estado de conexión TCP
12	`CMD_NET_MIRROR`	Activar mirror de paquetes (debug)
13	`CMD_UDP_HAS_DATA`	Verificar si hay datos UDP pendientes

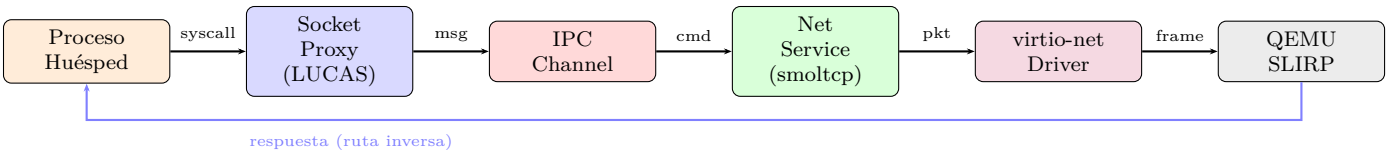

Figura 7.1: Ruta completa de una operación de red: desde la syscall del huésped hasta el backend SLIRP de QEMU, pasando por el proxy de sockets LUCAS, el canal IPC, el servicio de red con smoltcp, y el driver virtio-net.

7.1.2. Flujo de una Transferencia TCP

7.1.3. Transferencia TCP en Bloques

El transporte IPC de sotOS tiene un límite inherente de datos por mensaje: 40 bytes para envío (`regs[3..]` de 5 registros × 8 bytes) y 64 bytes para recepción (`regs[0..7]` de 8 registros × 8 bytes). Para transferencias TCP grandes (e.g., descargar una página web con `wget`), LUCAS fragmenta los datos en chunks de 64 bytes y los reensambla:

Algoritmo 20: TCP-Recv-Multi: Recepción TCP multi-chunk

Input: Conexión `conn_id`, buffer destino `buf`, tamaño `len`
Output: Bytes recibidos

```
total ← 0
while total < len do
    msg ← IPC-Call(CMD_TCP_RECV, conn_id, total)
    chunk_len ← msg.tag
    if chunk_len = 0 then
        salir ;                                   // EOF o no más datos
    Copiar mín(chunk_len, 64) bytes de msg.regs[0..7] a buf[total..]
    total += chunk_len
return total
```

7.1.4. Resolución DNS por UDP

El servicio de red resuelve nombres de dominio enviando consultas DNS por UDP al servidor configurado por DHCP (por defecto `10.0.2.3`, el DNS de SLIRP de QEMU). Las respuestas DNS mayores a 64 bytes se recuperan en múltiples chunks con un offset en el argumento 2.

7.2. Servicio VMM (Virtual Memory Manager)

El servidor VMM es el primer servicio lanzado y maneja todos los fallos de página del sistema. Su diseño es intencionalmente simple: un bucle infinito que espera notificaciones de fallo y las resuelve.

7.2.1. Manejo de Fallos de Página

El VMM se registra de dos maneras:

1. **Per-AS**: `fault_register_as(notify, init_as_cap)` para los fallos del proceso init.

2. **Global**: `fault_register(notify)` con `rsi=0`, que registra al VMM como manejador de respaldo para *cualquier* espacio de direcciones (incluyendo los clones CoW de procesos hijos).

Los tipos de fallo que el VMM maneja son:

1. **Demanda de paginación**: Página no presente → asignar marco, mapear como WRITABLE, reanudar hilo.

2. **CoW**: Escritura a página presente sin permiso de escritura (`code & 0x03` = `0x03`) → ejecutar CoW-Fault-Handler (algorithm 19).

3. **Violación NX**: Fetch de instrucción en página NX (`code & 0x11` = `0x11`) → cambiar permisos a R+X (sin WRITABLE para mantener W^X).

4. **Bucle infinito**: Si la misma dirección genera > 8 fallos consecutivos → inyectar SIGSEGV (señal 11) al hilo.

7.2.2. Registro Global de Fallos

Invariante 7.1 (Cobertura de fallos). Todo fallo de página en espacio de usuario es manejado por el VMM. Si un espacio de direcciones tiene un manejador per-AS registrado, se usa ese. De lo contrario, el manejador global (CR3 = 0) captura el fallo. No existen fallos de página de usuario sin manejador.

7.3. Servicio de Teclado

El driver de teclado PS/2 ejecuta como un proceso independiente que:

1. Registra un manejador de IRQ para IRQ 1 (teclado PS/2).

2. Lee los scancodes del puerto `0x60`.

3. Traduce scancodes a eventos de tecla (key down/up).

4. Escribe los caracteres resultantes en un ring buffer compartido en `0x510000` (dirección centralizada en `sotos_common`).

LUCAS lee del ring buffer cuando un proceso huésped ejecuta `read(0, ...)` (lectura de stdin, fd 0).

7.4. ObjectStore y VFS

7.4.1. Layout en Disco

El `ObjectStore` (versión 6) organiza el disco en regiones contiguas:

7.4.2. Entrada de Directorio

Cada entrada de directorio ocupa 128 bytes (4 por sector de 512 B):

Cuadro 7.2: Layout de sectores del ObjectStore v6. Cada sector tiene 512 bytes.

Sector(es)	**Región**	**Tamaño**	**Contenido**
0	Superblock	1 sector (512 B)	Magia, versión, contadores
1	WAL Header	1 sector (512 B)	Secuencia, entradas, committed
2–5	WAL Payload	4 sectores (2 KiB)	Copias de sectores a escribir
6–133	Bitmap	128 sectores (64 KiB)	524 288 bits = estado de cada sector de datos
134–2181	Directorio	2048 sectores	8192 entradas de 128 B
2182–2197	Refcount	16 sectores	8192 × `u8` (para CoW snapshots)
2198–2201	Snapshot Meta	4 sectores	Metadatos de hasta 4 snapshots
2202–10393	Snapshot Dir	4 × 2048 sectores	Copia del directorio por snapshot
10394–10905	Snapshot Bitmap	4 × 128 sectores	Copia del bitmap por snapshot
10906+	Datos	Variable	Contenido de los archivos

Offset	**Campo**	**Tipo/Tamaño**
0	`oid`	`u64` (8 B) — identificador único del objeto
8	`name`	`[u8; 48]` — nombre, terminado en null
56	`size`	`u64` (8 B) — tamaño del archivo en bytes
64	`sector_start`	`u32` (4 B) — primer sector de datos
68	`sector_count`	`u32` (4 B) — número de sectores ocupados
72	`flags`	`u32` (4 B) — bit 0 = directorio
76	`created_time`	`u64` (8 B) — timestamp Unix (segundos)
84	`modified_time`	`u64` (8 B) — timestamp Unix
92	`accessed_time`	`u64` (8 B) — timestamp Unix
100	`parent_oid`	`u64` (8 B) — OID del directorio padre
108	`permissions`	`u32` (4 B) — permisos Unix (rwxrwxrwx)
112	`owner_uid`	`u16` (2 B) — UID del propietario
114	`owner_gid`	`u16` (2 B) — GID del propietario
116	`_pad`	`[u8; 8]` — relleno a 128 bytes

Nota sobre alineación: El campo `oid` (offset 0) + `name` (48 B) + `size` (8 B) suman 64 bytes, alineando `sector_start` en una frontera de 64 B. La estructura completa es de 128 bytes exactos, verificado en tiempo de compilación con `const_assert`.

7.4.3. Write-Ahead Logging

El WAL (Write-Ahead Log) garantiza atomicidad de las escrituras. Cada transacción puede afectar hasta 4 sectores.

Algoritmo 21: WAL-Write: Escritura atómica con WAL

Input: Conjunto de pares $\{(\texttt{target}_i, \texttt{data}_i)\}_{i=1}^{k}$ con $k \leq 4$

```
// Fase 1: Escribir payload al WAL
for i ← 1 to k do
    Escribir data_i en sector SECTOR_WAL_PAYLOAD + i − 1
// Fase 2: Escribir header con committed=0
hdr.seq += 1
hdr.entry_count ← k
hdr.targets[0..k − 1] ← sectores destino
hdr.committed ← 0
Escribir hdr en sector 1
// Fase 3: Escribir datos a sus destinos finales
for i ← 1 to k do
    Escribir data_i en sector target_i
// Fase 4: Marcar como committed
hdr.committed ← 1
Escribir hdr en sector 1
```

Teorema 7.1 (Atomicidad del WAL)**.** *Sea $T = \{(\texttt{target}_i, \texttt{data}_i)\}_{i=1}^{k}$ una transacción ejecutada por WAL-Write. En caso de fallo (corte de energía, reset) en cualquier punto de la ejecución:*

1. *Si el fallo ocurre antes de la Fase 4 (`committed` aún $= 0$): los sectores destino pueden estar parcialmente escritos, pero el WAL aún contiene los datos correctos en el payload. En la recuperación, se puede repetir la Fase 3 completa.*

2. *Si el fallo ocurre después de la Fase 4: la transacción está completamente persistida.*

En ambos casos, el sistema de archivos recupera un estado consistente.

Demostración. Consideramos los cuatro puntos de fallo posibles:

Fallo durante Fase 1 (escritura del payload al WAL): El header aún no se ha escrito, por lo que en recuperación `hdr.entry_count` $= 0$ o el `seq` no ha cambiado. No hay transacción pendiente. Los sectores destino no fueron modificados. Estado: consistente (transacción no ocurrió).

Fallo durante Fase 2 (escritura del header): El header puede estar corrupto o tener `committed` $= 0$ con `entry_count` > 0 pero magia incorrecta ($\neq$ `0x57414C48`). En recuperación, se verifica la magia; si es inválida, se ignora. Si es válida pero `committed` $= 0$, se re-ejecuta Fase 3 con los datos del payload (que están completos desde Fase 1). Estado: consistente.

Fallo durante Fase 3 (escritura a destinos): Algunos sectores destino pueden estar escritos y otros no. Pero `committed` $= 0$, y el payload completo está en el WAL. En recuperación, se re-ejecuta Fase 3 completa (idempotente: escribir el mismo dato al mismo sector). Estado: consistente.

Fallo durante Fase 4 (marcar committed): Si la escritura del header con `committed` $= 1$ no se completó, en recuperación `committed` será 0, y se repite Fase 3 (los datos ya están en los destinos, pero la repetición es inofensiva). Si se completó, la transacción está persistida. En ambos sub-casos: consistente.

En todos los casos, el invariante se mantiene: o la transacción se aplica completamente, o se puede completar durante la recuperación. □

Capítulo 8

Arranque y Configuración

Este capítulo describe el proceso completo de arranque de sotOS, desde el encendido de la máquina (real o virtual) hasta que el shell LUCAS está listo para recibir comandos.

8.1. Bootloader: Limine (BIOS, GPT+FAT32)

sotOS utiliza el bootloader Limine v8 en modo BIOS. El disco de imagen tiene:

- Tabla de particiones GPT con una partición FAT32.
- El binario de Limine instalado en el MBR y en los sectores de boot.
- La configuración en `/boot/limine.conf`.

La configuración de Limine es mínima:

```
serial: yes
timeout: 0

/sotOS
    protocol: limine
    resolution: 1024x768x32
    kernel_path: boot():/boot/sotos-kernel
    module_path: boot():/boot/initrd.img
```

Listing 8.1: Contenido de `boot/limine.conf`.

La directiva `serial: yes` habilita la salida de Limine por el puerto serial COM1, lo que permite ver los mensajes del bootloader en la terminal QEMU antes de que el kernel tome control.

8.2. Secuencia de Arranque del Kernel

La función `kmain()` ejecuta los siguientes pasos en orden estricto:

1. **Inicializar serial (COM1)**: Primera instrucción. Todas las aserciones y mensajes de debug dependen de la salida serial. `kprintln!("sotOS v0.1.0 -- microkernel booting...")`.
2. **GDT**: Instalar la Global Descriptor Table con segmentos de kernel (Ring 0) y usuario (Ring 3). Cargar el Task State Segment (TSS) con pilas de interrupción (IST) para #DF, #GP, y misc.

3. **SSE**: Habilitar SSE/SSE2 en CR0 y CR4. Crítico para binarios musl/glibc que usan instrucciones como `movaps`. Se aplica al BSP aquí y a cada AP durante su inicialización.
4. **IDT**: Instalar la Interrupt Descriptor Table con handlers para excepciones (#PF, #GP, #DF) e IRQs de hardware.
5. **Asignador de marcos**: Inicializar el bitmap desde el mapa de memoria del bootloader (vía HHDM).
6. **Slab allocator**: Inicializar el heap del kernel (slabs de 64, 128, 256, 512 bytes).
7. **Per-CPU + TSS**: Asignar estructura `PerCpu` para el BSP, recargar GDT con TSS per-CPU (pilas IST únicas por CPU).
8. **Capacidades + Planificador**: Inicializar tabla de capacidades, pool de objetos, MLQ de 4 niveles. Configurar MSRs de SYSCALL/SYSRET.
9. **LAPIC**: Inicializar el Local APIC, calibrar el timer (∼100 Hz periódico), enmascarar el PIT (obsoleto).
10. **Proceso init**: Cargar el ELF del initrd, crear primer espacio de direcciones de usuario, spawning de los servicios (VMM, kbd, net).

Después de estos pasos, el kernel habilita interrupciones y entra en el bucle idle (`hlt` en un loop). Las interrupciones del LAPIC timer causan cambios de contexto preemptivos al planificador.

8.3. Bootstrapping del Proceso Init

El proceso `init` (cargado desde el initrd CPIO) es el primer código de espacio de usuario. Su bootstrapping incluye:

1. Leer la página `BootInfo` en `0xB00000` para obtener las capacidades raíz (endpoints, AS caps, frame caps).
2. Inicializar el framebuffer serial (consola de texto vía COM1).
3. Registrar el servicio `ïnit"` en el registro de servicios del kernel.
4. Spawning de procesos hijo:
   - **VMM**: Recibe cap de AS del init + cap de notificación.
   - **Keyboard**: Recibe cap de IRQ para IRQ 1.
   - **Network**: Recibe cap de PCI para escaneo de dispositivos.
5. Montar el `ObjectStore` sobre el disco virtio-blk.
6. Inyectar el sysroot virtual (`/lib`, `/bin`, `/etc`, `/tmp`, `/proc`, `/sys`).
7. Forjar el vDSO en `0xB80000`.
8. Ejecutar pruebas de boot (hello-linux, musl, busybox, etc.).
9. Lanzar el shell LUCAS.

Cuadro 8.1: Mapa completo del espacio de direcciones del proceso init. Todas las direcciones están en la mitad inferior del espacio virtual (usuario).

Dirección	Tamaño	Contenido
`0x200000`	Variable	ELF `.text` (init binary)
`0x510000`	4 KiB	Ring buffer del teclado (compartido)
`0x520000`	4 KiB	Ring buffer del mouse (compartido)
`0x900000`	16 KiB	Stack del init (4 páginas, con ASLR)
`0xA00000`	4 KiB	SPSC ring buffer + stack del producer
`0xB00000`	4 KiB	BootInfo (solo lectura)
`0xB80000`	4 KiB	vDSO (R+X, ELF forjado)
`0xC00000+`	Variable	Páginas MMIO de virtio (UC)
`0xD00000+`	1.25 MiB	ObjectStore + VFS (320 páginas)
`0xE50000`	16 KiB	Stack del huésped LUCAS (4 páginas)
`0xE60000`	64 KiB	Stack del handler LUCAS (16 páginas)
`0xE70000`	4 KiB	VirtioBlk storage (1 página)
`0xE80000+`	Variable	Stacks de procesos hijo (0x2000 c/u)
`0x1000000`	Variable	Shell ELF
`0x2000000`	1 MiB	Heap `brk`
`0x3000000`	Variable	Región `mmap`
`0x4000000`	Variable	Framebuffer (UC, si está presente)
`0x5000000`	512 KiB	Buffer de spawn (128 páginas)
`0x5200000`	128 KiB	Buffer de dynamic linking (32 páginas)
`0x5400000`	512 KiB	Buffer de exec (128 páginas)
`0x6000000`	Variable	`INTERP_LOAD_BASE` (ld.so)
`0xA000000`	Variable	`INTERP_BUF_BASE`

8.4. Layout del Espacio de Direcciones

8.5. Configuración de QEMU

El sistema de build utiliza `just` (make alternativo) con los siguientes targets principales:
La línea de QEMU para `just run-net` (el target más completo) es:

```
qemu-system-x86_64 \
    -drive format=raw,file=target/sotos.img \
    -drive if=none,format=raw,file=target/disk.img,id=disk0 \
    -device virtio-blk-pci,drive=disk0,disable-modern=on \
    -netdev user,id=net0,dns=8.8.8.8,
            hostfwd=udp::5555-:5555,hostfwd=tcp::7777-:7 \
    -device virtio-net-pci,netdev=net0,disable-modern=on \
    -serial stdio -no-reboot -m 1024M
```

Listing 8.2: Comando QEMU para `just run-net`.

Aspectos relevantes de la configuración:

- `disable-modern=on`: Fuerza virtio legacy (no MMIO moderno), compatible con los drivers de SOTOS.
- `hostfwd`: Reenvía puertos del host al guest (UDP 5555 para pruebas, TCP 7777 para echo server).

Cuadro 8.2: Targets de `just` para ejecución en QEMU.

Target	Descripción
`just build`	Compilar el kernel (`cargo build -package sotos-kernel`)
`just run`	Build + imagen + QEMU (serial a terminal, sin display, 1 CPU, 1 GiB RAM)
`just run-gui`	Como `run` pero con display SDL + virtio-blk
`just run-net`	Con virtio-net (SLIRP) + virtio-blk (ObjectStore en disco)
`just run-smp`	Con 4 CPUs (`-smp 4`, experimental por race en scheduler)
`just debug`	Con GDB server (`-s -S`, conectar con `target remote :1234`)

- `dns=8.8.8.8`: Configura el DNS del backend SLIRP.
- `-m 1024M`: 1 GiB de RAM (incrementado desde 256 MiB original para soportar ObjectStore + procesos dinámicos).
- `-serial stdio`: Salida serial del kernel directamente a la terminal.
- `-no-reboot`: QEMU sale en lugar de reiniciar ante triple fault.

8.6. Resumen de la Parte II

Esta segunda parte ha cubierto los componentes de espacio de usuario que transforman las cinco primitivas del micronúcleo en un sistema operativo funcional:

1. **Gestión de memoria** (chapter 5): Asignador bitmap con $\mathcal{O}(1)$ amortizado, tablas de páginas de cuatro niveles, y Copy-on-Write con pruebas formales de aislamiento y cotas de memoria.
2. **LUCAS** (chapter 6): Capa de compatibilidad Linux con 127+ syscalls emuladas, soporte completo de fork/exec/wait, señales asíncronas, y un vDSO forjado para acceso rápido al tiempo.
3. **Servicios del sistema** (chapter 7): Red TCP/IP (smoltcp, 13 comandos IPC), VMM con CoW, teclado PS/2, y ObjectStore transaccional con WAL y snapshots.
4. **Arranque** (chapter 8): Limine BIOS, secuencia de 10 pasos del kernel, bootstrap de init, y configuración de QEMU.

Bibliografía

[1] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, 1999.

[2] J. Liedtke, "On micro-kernel construction," *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 237–250, 1995.

[3] J. S. Shapiro, J. M. Smith, and D. J. Farber, "EROS: A fast capability system," *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 170–185, 1999.

[4] G. Klein et al., "seL4: Formal verification of an OS kernel," *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 207–220, 2009.

[5] D. R. Engler, M. F. Kaashoek, and J. O'Toole, "Exokernel: An operating system architecture for application-level resource management," *Proceedings of the 15th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 251–266, 1995.

[6] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[7] H. M. Levy, *Capability-Based Computer Systems*, Digital Press, 1984.

[8] M. Herlihy, "Wait-free synchronization," *ACM Transactions on Programming Languages and Systems*, vol. 13, no. 1, pp. 124–149, 1991.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.